MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING

SOFTWARE DEVELOPMENT THESIS PROJECT
COURSE: CT250H

Thesis title

DESIGN AND IMPLEMENTATION OF A HUMAN-IN-THE-LOOP AI-ASSISTED RECRUITMENT AUTOMATION PLATFORM FOR CV-JD EVALUATION AND RECOMMENDATION IN IT

Student
Pham Huu Hung - B2203557

Can Tho, August 2026

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING

SOFTWARE DEVELOPMENT THESIS PROJECT
COURSE: CT250H

Thesis title

DESIGN AND IMPLEMENTATION OF A HUMAN-IN-THE-LOOP AI-ASSISTED RECRUITMENT AUTOMATION PLATFORM FOR CV-JD EVALUATION AND RECOMMENDATION IN IT

Supervisor
Ph.D. Nguyen Thanh Khoa

Student
Pham Huu Hung - B2203557

Can Tho, August 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Ph.D. Nguyen Thanh Khoa for the guidance, constructive feedback, and continuous support provided throughout this thesis. The supervisor's academic and technical advice played an important role in shaping the research direction and improving the quality of this work.

I am also grateful to the lecturers and staff of the College of Information and Communication Technology, Can Tho University, for providing the academic foundation and learning environment necessary for the completion of this project. My thanks are extended to everyone who contributed feedback during the design, implementation, and evaluation of the CareerFit system.

Finally, I would like to thank my family and friends for their encouragement and support throughout my studies. Although every effort has been made to ensure the accuracy and quality of this thesis, limitations may remain, and constructive comments are sincerely appreciated.

Can Tho, August 2026

Pham Huu Hung

DECLARATION OF ORIGINALITY

I hereby declare that this thesis, titled “Design and Implementation of a Human-in-the-Loop AI-Assisted Recruitment Automation Platform for CV-JD Evaluation and Recommendation in IT,” is my original work completed under the supervision of Ph.D. Nguyen Thanh Khoa. The implementation, experiments, data analysis, and conclusions presented in this report are reported truthfully within the stated scope.

All ideas, publications, standards, software documentation, figures, and other materials obtained from external sources are acknowledged through citations or source notes. I accept full responsibility for the accuracy, academic integrity, and originality of the submitted work.

Can Tho, August 2026
Student

Pham Huu Hung

SUPERVISOR'S COMMENTS

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Can Tho, August 2026
Supervisor

Ph.D. Nguyen Thanh Khoa

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Full Term	Meaning / Explanation
AI	Artificial Intelligence	Computer systems designed to perform tasks commonly associated with human intelligence
API	Application Programming Interface	A defined interface through which software components communicate
CV	Curriculum Vitae	A document describing a candidate's qualifications and experience
E2E	End-to-End	Testing of a complete workflow across integrated components
HITL	Human-in-the-Loop	Automation in which humans retain oversight or approval authority
JD	Job Description	A description of a job's responsibilities and requirements
JWT	JSON Web Token	A compact token format used for authentication and authorization
MRR	Mean Reciprocal Rank	A ranking metric based on the position of the first relevant item
nDCG	Normalized Discounted Cumulative Gain	A ranking metric that accounts for relevance and position
NLP	Natural Language Processing	Methods for processing and analyzing human language
OCR	Optical Character Recognition	Extraction of machine-readable text from images or scanned documents
RBAC	Role-Based Access Control	Authorization based on assigned user roles
REST	Representational State Transfer	An architectural style for networked APIs
TF-IDF	Term Frequency–Inverse Document Frequency	A weighting method for representing text terms

Abbreviation	Full Term	Meaning / Explanation
UAT	User Acceptance Testing	Validation of whether the system satisfies intended user workflows

ABSTRACT

Background: Recruitment in the information technology sector requires candidates and recruiters to compare many types of information from curricula vitae and job descriptions. Most job portals support posting, searching, and manual applications, while automated screening tools may not clearly explain how a score is calculated or how an automated action is controlled.

Objectives: This thesis designs and implements CareerFit IT AutoPilot, a web-based recruitment platform that combines a job portal, CV-job description matching, personalized job recommendations, policy-based automation, and Human-in-the-Loop interaction in one auditable workflow.

Methods: Recruitment text is normalized and represented with Term Frequency-Inverse Document Frequency vectors. Cosine similarity ranks CV-job pairs and candidate-job recommendations, while Rocchio feedback updates learned representations from explicit feedback. A separate skill-transfer model explains Potential candidates without changing the direct cosine score. AutoFit checks consent and policy limits before an action. The platform also has a report moderation flow for suspicious Jobs and CVs. CareerFit uses a Spring Boot backend, a React and TypeScript frontend, and PostgreSQL with Flyway migrations.

Results: On August 7, 2026, the refreshed backend suite passed 141 of 141 tests across 35 suites. TypeScript checking, ESLint, the production frontend build, and the bundle check also passed. All 46 Chrome workflow, contract, and resilience tests passed against the integrated application, including Job/CV reporting and administrator moderation cases. In the controlled synthetic benchmark, nDCG@5 increased from 0.037737 to 0.837737 after Rocchio feedback. These results support a tested academic prototype, but they do not prove production readiness or real-world hiring effectiveness.

Keywords: recruitment automation; CV-job matching; recommendation system; Human-in-the-Loop; Rocchio feedback; explainable automation.

INTRODUCTION

Problem Statement

IT recruitment requires candidates and recruiters to compare many details, including technical skills, experience, language, location, and working conditions. Candidates must identify suitable vacancies, while recruiters must review CVs and decide which applicants deserve closer attention. As the number of jobs and profiles grows, doing this manually becomes slow and inconsistent [10].

Most job portals support posting, searching, and applying, but they do not fully explain why a job fits a candidate or why one applicant ranks above another. CareerFit therefore treats CV–JD matching and personalized recommendation as related but separate tasks. The first compares a specific CV and job; the second prioritizes jobs from a candidate's broader profile and preferences.

Automation can reduce repetitive work, but a similarity score should not directly trigger an important action. User consent, application state, previous interactions, thresholds, quotas, and special conditions also matter. CareerFit uses a Human-in-the-Loop design so users can review important actions and administrators can trace what happened.

This thesis develops CareerFit IT AutoPilot, an IT-focused web platform that combines job discovery, reviewed CV and JD processing, matching, recommendation, Rocchio feedback, AutoFit policies, secure email actions, recruiter talent discovery, and audit records. The system helps users make decisions but does not replace recruitment judgment.

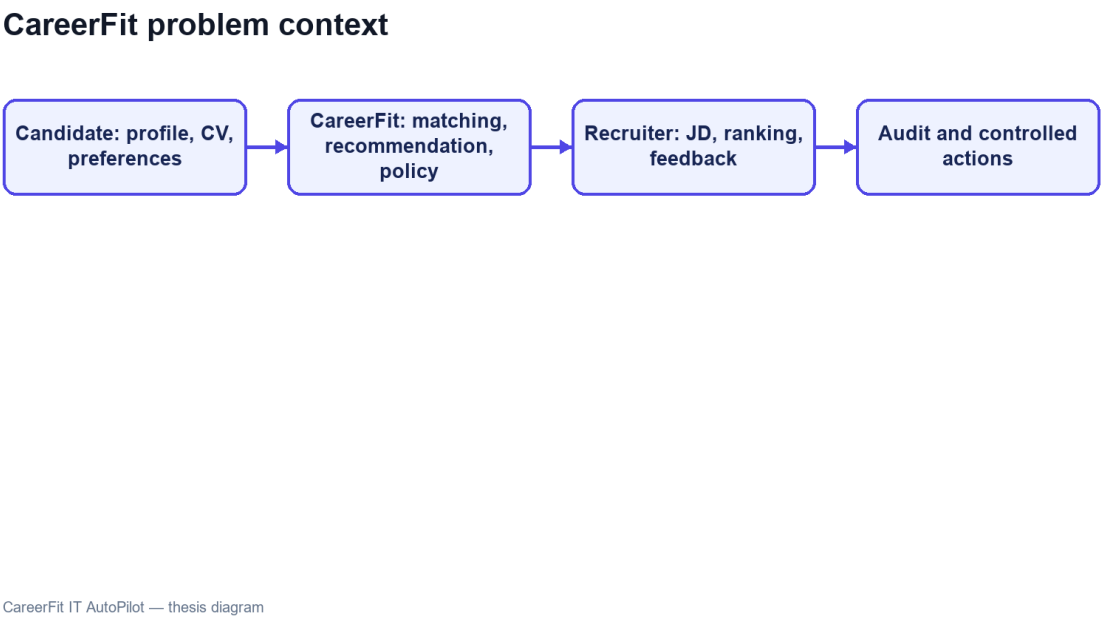


Figure 1.1. CareerFit problem context and principal information flows

Background and Related Work

The first motivation is to reduce fragmentation. Candidates often move between job search, CV management, application history, and email, while recruiters switch between vacancy management, applicant review, candidate discovery, and reporting. CareerFit brings these activities into one role-based system and uses email as an additional action channel.

The second motivation is explainability. TF-IDF and cosine similarity do not capture meaning as deeply as modern embedding models, but their terms and weights can be inspected. This makes them suitable for an academic baseline in which the scoring process, limitations, and effect of feedback must be demonstrated clearly.

The third motivation is controlled adaptation and automation. AutoFit places policy checks between a score and an action, considering consent, thresholds, previous interactions, cooldown, quota, time zone, and quiet hours. Rocchio feedback then provides a transparent way to adjust later rankings from explicit positive and negative judgments without overwriting the original representation.

Related work includes classical lexical retrieval, learned resume-job representations, explainable recommendation, and human-centered governance. Lexical methods provide inspectable weights and matching reasons, while recent dense approaches improve semantic representation but require stronger data and evaluation assumptions [6]-[8]. Research on algorithmic hiring and explainable recommendation also shows that model performance, fairness, user control, and

explanation quality must be evaluated separately [9], [10], [14]. CareerFit addresses a narrower integration gap by combining portal workflows, explainable scoring, feedback learning, policy-controlled actions, and audit records in one academic prototype.

Research Objectives

Overall Objective

The overall objective of this thesis is to design, implement, and evaluate a web-based Human-in-the-Loop recruitment automation platform for the IT domain. The platform is intended to support CV–job description evaluation, personalized job recommendation, feedback-based adaptation, and policy-driven actions while preserving user control, explainability, security, and auditability.

Specific Objectives

To achieve the overall objective, the thesis defines the following specific objectives:

- Design a role-based web platform for guests, candidates, recruiters, and administrators, with protected workflows and a content-reporting path for suspicious Jobs and CVs.
- Provide candidate profile and CV management, including upload, manual drafts, OCR-assisted extraction, section review, user correction, confirmation, validation, and quality feedback before matching.
- Implement separate CV–JD matching and candidate-profile recommendation workflows using a shared text-normalization, TF-IDF, and cosine-similarity pipeline.
- Present normalized scores, relevance labels, matching reasons, validation signals, and a separate Potential explanation based on transferable skills, skill families, shared foundations, and seniority compatibility.
- Integrate explicit user feedback and apply the Rocchio relevance-feedback method to update learned representations and recompute rankings reproducibly.
- Implement AutoFit policies that convert matching results into controlled outcomes according to consent, thresholds, interaction state, quota, cooldown, time-zone, and quiet-hour rules.
- Support expiring email-action links with hashed token storage, a non-mutating confirmation page, POST execution, action-state tracking, and audit logging.
- Evaluate the algorithmic behavior and system workflows using reproducible experiments, automated tests, scripted acceptance scenarios, and security and reliability checks appropriate to the project scope.

Research Subjects and Scope

Research Subject

The research subject is the IT recruitment workflow supported by a web platform, including CV and Job-description text, lexical ranking and recommendation, explicit feedback signals, policy-controlled automation, and auditable user and system actions.

Research Scope

This thesis focuses on an academic prototype for IT recruitment. Its functional scope includes a public job portal; authenticated Candidate, Recruiter, and Administrator workspaces; reviewed CV and profile management; recruiter company profiles; Job drafts and publishing; CV–JD matching; personalized recommendation; applications; Talent Pool bookmarks and invitations; explicit feedback; AutoFit policies; notifications; analytics; content reporting and administrator moderation; and audit records.

The data-processing scope includes extraction from supported CV documents, image preprocessing and OCR fallback, OCR cleanup, section review, bilingual-oriented text normalization, TF-IDF vectorization, cosine-similarity scoring, a versioned skill-transfer Potential assessment, relevance labeling, and Rocchio feedback updates. PostgreSQL is the main data store, and Flyway manages database changes. The React and TypeScript frontend calls REST APIs provided by the Spring Boot backend.

The thesis does not attempt to implement a complete enterprise applicant tracking system. Interview scheduling, offer management, payroll, and full employee lifecycle management are outside the core scope. The system does not automatically submit applications to external third-party job boards, and it does not use a large language model to make unrestricted recruitment decisions. Distributed microservices, large-scale message brokers, and semantic embedding models may be discussed as future extensions but must not be presented as implemented core components unless they are added and verified before the final report is submitted.

The evaluation scope is limited. Controlled or synthetic datasets can show whether Rocchio changes rankings in the expected direction, but they cannot prove recruitment quality in a real production setting. The source and purpose of scraped, seeded, or synthetic data must be stated. Claims about usability, performance, security, or production readiness are limited to the test environment, participants, procedures, and evidence available during the final evaluation.

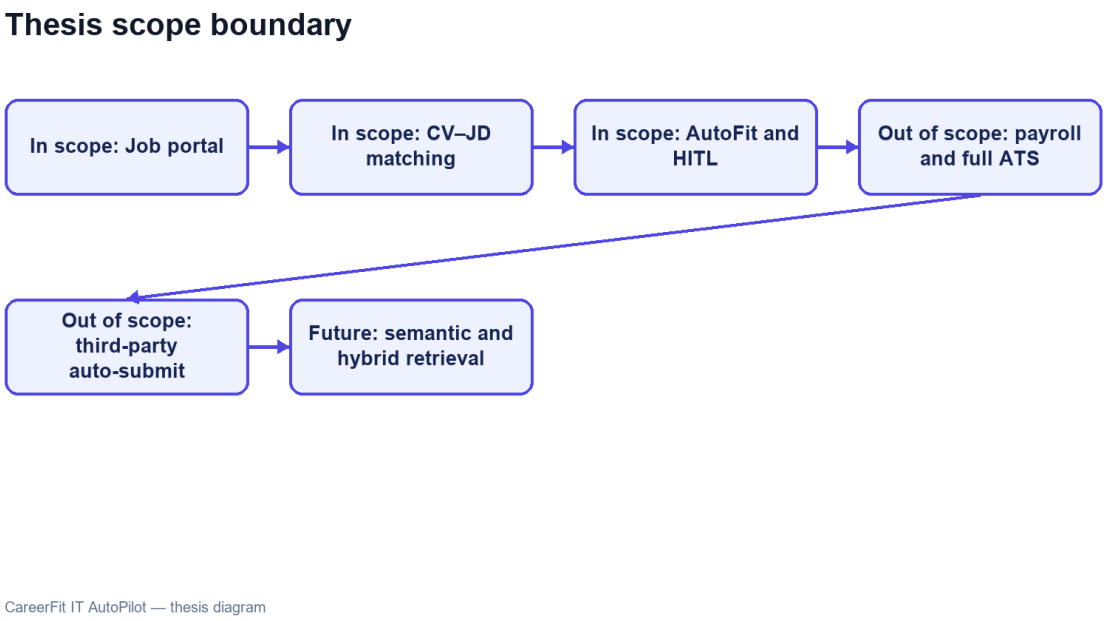


Figure 1.2. Scope boundary of the CareerFit thesis

Research Methods and Content

Research Methods

This thesis follows an applied software-engineering research approach. The work combines requirement analysis, architecture and data design, incremental implementation, a controlled algorithm experiment, automated backend and browser testing, runtime observation, and threats-to-validity analysis. Implementation claims are checked against source code, migrations, API contracts, logs, test reports, and generated evaluation artifacts.

Research Content

The research models CareerFit as a Perception-Decision-Action-Learning-Audit workflow. It first identifies recruitment roles, requirements, and control boundaries; then designs and implements the data, matching, recommendation, feedback, automation, email action, and audit modules. Finally, the study evaluates algorithm behavior, software correctness, browser workflows, authorization, runtime health, and local latency within the stated experimental scope.

Key Contributions

The principal contribution of this thesis is the design and implementation of an integrated, controlled recruitment workflow rather than the invention of a new machine-learning model. The contributions are summarized as follows:

- An integrated recruitment platform that combines public job discovery, reviewed CV processing, CV–JD matching, personalized recommendation, application management, Recruiter Jobs, Talent Pool, and a report-moderation workflow.
- A transparent scoring design that keeps the TF-IDF/cosine direct-match score separate from the versioned Potential assessment and explains transferable skills, shared foundations, and seniority checks.
- A feedback-learning workflow based on the Rocchio method, designed to update learned representations from explicit feedback and to support reproducible recomputation rather than uncontrolled cumulative modification.
- A policy-driven AutoFit layer that separates relevance scoring from business actions and applies Human-in-the-Loop controls, user consent, operational constraints, and previous interaction states.
- An actionable email mechanism with expiring, hashed tokens, a non-mutating GET confirmation page, POST redemption, and action-state tracking.
- A traceable evaluation structure that distinguishes algorithm experiments from functional, integration, acceptance, performance, reliability, and security evidence, while explicitly documenting threats to validity.

Thesis objective	Implemented contribution	Evaluation evidence
Analyze IT recruitment workflows	Role-specific requirements and Human-in-the-Loop boundaries	Use cases and traceability matrix
Implement CV–JD matching	TF-IDF, cosine scoring, explanations, and persisted Matchings	Algorithm benchmark and backend tests
Learn from feedback	Rocchio updates followed by scheduled recomputation	Baseline-versus-learned ranking metrics
Support controlled automation	AutoFit policy, quota, cooldown, notification, and audit controls	Security tests and P0 E2E flows
Deliver an integrated platform	Spring Boot, React/TypeScript, PostgreSQL, and Flyway	Build, runtime health, and browser evidence

Table 1.1. Mapping of objectives, contributions, and evidence

Thesis Structure

The report contains an Introduction, four chapters, and a Conclusion. Chapter 1 defines the problem, stakeholders, requirements, and use cases. Chapter 2 presents the theoretical background and solution design. Chapter 3 explains the

implementation. Chapter 4 presents the evaluation method, evidence, and threats to validity. The Conclusion discusses achievements, limitations, and future work.

MAIN CONTENT

CHAPTER 1. PROBLEM DESCRIPTION AND REQUIREMENTS

1.1 System Analysis Context

CareerFit is designed as a role-based system for IT recruitment. Its boundary includes the React frontend, Spring Boot backend, PostgreSQL database, CV storage, background scheduler, and optional email provider. External job boards, interview scheduling, offers, payroll, and unrestricted automated hiring remain outside the implemented scope.

The central design distinction is between evidence, business state, and action. A matching record contains evidence about the similarity between one CV and one job. An application record represents a candidate's participation in a recruitment process. An automation policy represents consent and operational preferences. These concepts must remain separate: a high similarity score must not silently become an application, and an application status must not be interpreted as a relevance label.

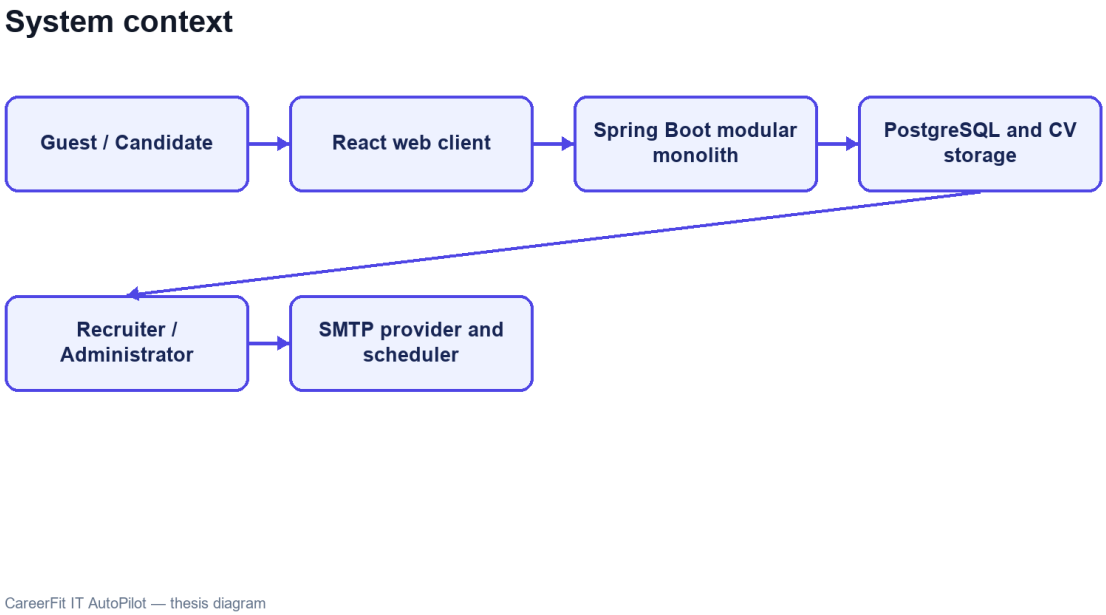


Figure 1.3. CareerFit system context

1.2 Stakeholders and Actors

CareerFit serves four interactive roles and two supporting actors. Guest is a frontend state rather than a persisted account role. Candidate, Recruiter, and Administrator are persisted roles in user_account. Background processing and the mail provider are non-human actors that execute scheduled work and deliver messages.

Table 1.2. Actors and responsibilities

Actor	Primary responsibilities	Access boundary
Guest	Browse public Jobs, details, employers, urgent items, and public market analytics	Public GET operations only; authentication is required for personalized data or applications
Candidate	Maintain profile/CVs; view matches; apply; give feedback; configure AutoFit; report an active Job	Candidate-owned resources and Candidate/report APIs
Recruiter	Maintain company/Jobs; review applicants and Talent Pool; invite Candidates; report a visible CV	Recruiter APIs, owned Jobs, and CVs visible through an owned Job
Administrator	Monitor users, Jobs, audit records, and report cases; dismiss reports or ban reported Jobs/CVs	Administrative APIs only; moderation actions are audited
Background scheduler	Recompute Matchings, send reminders, clean actions, and run eligible automation	Internal service calls under system control
Mail provider	Deliver notification and signed action emails generated by the backend	SMTP boundary; it does not own CareerFit business state

Stakeholder needs sometimes conflict. Candidates want relevant opportunities, privacy, and control over automatic applications. Recruiters want efficient prioritization without losing access to applicants who do not rank highly. Administrators need visibility and recoverability without unrestricted exposure of CV content. The design therefore combines role authorization, ownership checks, policy settings, validation, explicit statuses, and audit records rather than relying on a single ranking score.

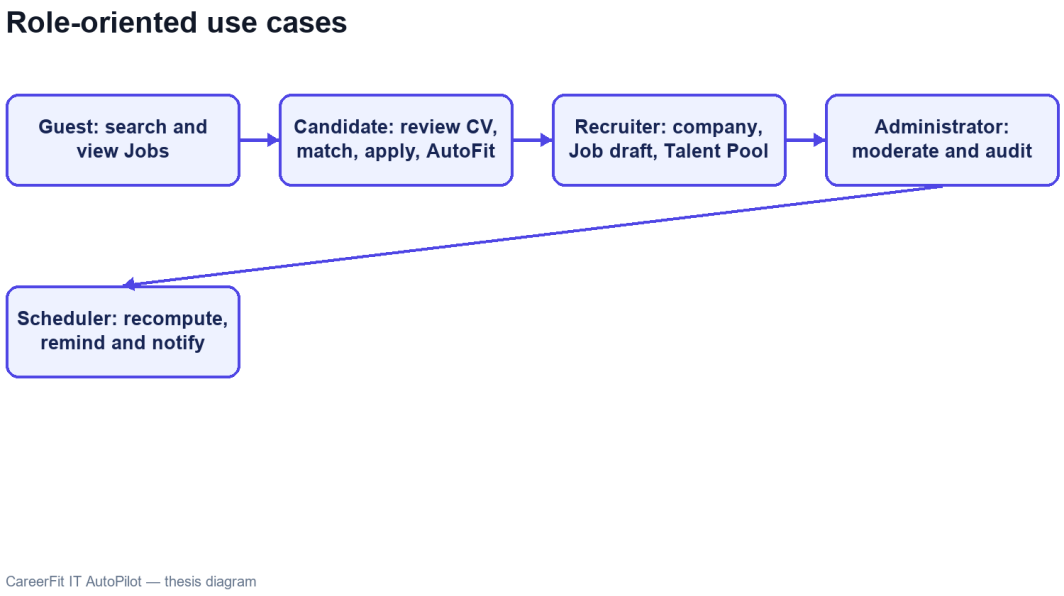


Figure 1.4. CareerFit use-case overview

1.3 Functional Requirements

The detailed Software Requirements Specification contains individual requirement identifiers. For the thesis, they are consolidated into functional groups that map to implemented modules and observable workflows.

Table 1.3. Consolidated functional requirements

Group	Required capabilities	Main actor
Authentication and account	Registration, email/password login, current-account lookup, role enforcement, and settings	Candidate, Recruiter, Administrator
Public Job portal	Server-side search, filters, urgent state, pagination, Job details, employers, and public analytics	Guest and authenticated users
Candidate profile and CV	Profile/portfolio, CV drafts, upload, OCR cleanup, review/edit/confirm, default CV, and quality signals	Candidate

Group	Required capabilities	Main actor
Job and employer management	Company profile; JD quality; Job draft, publish, update, urgent/deadline fields, status, export, and counts	Recruiter
Matching and recommendation	Direct CV–JD score, reasons, Potential assessment, Candidate cards, ranking, recommendations, and similar Jobs	Candidate and Recruiter
Application and Talent Pool	Apply, history, withdrawal, applicant review, Potential discovery, bookmark, invitation, and status update	Candidate and Recruiter
Feedback and automation	Rocchio feedback, recomputation, account policy, quota, cooldown, quiet hours, auto-apply, and reminders	Candidate and Scheduler
Content reporting	Candidate reports active Job; Recruiter reports visible CV with owned Job context; prevent duplicate pending reports	Candidate and Recruiter
Administration	Report queue/detail, dismiss or ban Job/CV, user/Job monitoring, notifications, email actions, and audit logs	Administrator

The implementation exposes these groups through REST-oriented endpoints. Examples include /api/jobs/search, /api/cv/upload, /api/cv/{cvId}/review, /api/matches/me/cards, /api/recommendations/jobs, /api/applications, /api/recruiter/jobs/{jobId}/applicants, /api/automation/policy, /api/reports/jobs/{jobId}, /api/reports/cvs/{cvId}, /api/admin/reports, and /api/admin/audit-logs. Chapter 3 explains the main behavior.

1.4 Non-Functional Requirements

Non-functional requirements constrain how the workflows must operate. They are expressed as design targets; Chapter 4 must distinguish targets from properties verified by measurement.

Table 1.4. Non-functional requirements and design responses

Quality attribute	Requirement	Design response
Security	Protected resources must require authenticated and authorized access	Stateless Spring Security filter chain, JWT processing, role rules, ownership checks, BCrypt password hashing, CORS allow-list, CSP, and HSTS configuration
Privacy	CV and account data must not be public or unnecessarily logged	Candidate-scoped APIs, local protected storage boundary, selective DTOs, and audit metadata rather than full document content
Reliability	Repeated or failed operations should not create inconsistent domain state	Database constraints, transactions, unique keys, optimistic version columns, idempotency checks, explicit statuses, and retry-aware background work
Performance	Public queries and ranking views should remain responsive for the academic dataset	Database indexes, pagination, stored vectors, background scoring, and bounded result sets
Maintainability	Domain changes should remain localized and database evolution reproducible	Modular package structure, controller–service–repository separation, DTOs, configuration properties, Flyway migrations, and automated tests
Usability	Users must understand loading, errors, empty results, score context, and available actions	Role-specific pages, normalized labels and reasons, validation signals, and explicit action controls

Quality attribute	Requirement	Design response
Auditability	Significant automated or administrative events should be reconstructable	audit_log, notification delivery records, email-action state, timestamps, actor/source metadata, and administrative views
Explainability	The system should expose evidence without presenting a score as a hiring probability	Lexical term/skill reasons, separate labels and application states, policy settings, and documented limitations

Security and auditability are defense-in-depth properties. A role rule at the URL layer does not replace ownership verification in a service, and an audit row does not make an unsafe action safe. Similarly, a database index is a performance design decision, not proof of latency under load. The evaluation chapter must test each critical claim in the environment in which it is reported.

1.5 Use-Case Analysis

1.5.1 Candidate Uploads a CV and Receives Matches

Table 1.5. Use case - Candidate uploads a CV and receives matches

Field	Description
Use-case ID	UC-01
Primary actor	Candidate
Preconditions	The Candidate account is active, authenticated, and has permission to manage its own CVs.
Trigger	The Candidate uploads a supported document or saves manual CV content.
Main flow	Validate and extract the source; preprocess and OCR when needed; create review sections; show warnings; let the Candidate edit and confirm; normalize and vectorize; score active Jobs; persist unique Matchings; return ordered cards.
Alternative/exception flows	Save manual work as DRAFT; reject unsupported or unsafe content; record extraction failure; keep REVIEW_REQUIRED until the Candidate confirms; show an empty state when no Job is eligible.
Postconditions	The CV remains DRAFT/REVIEW_REQUIRED, reaches SCORING_DONE after confirmation, or ends in FAILED with a visible reason.

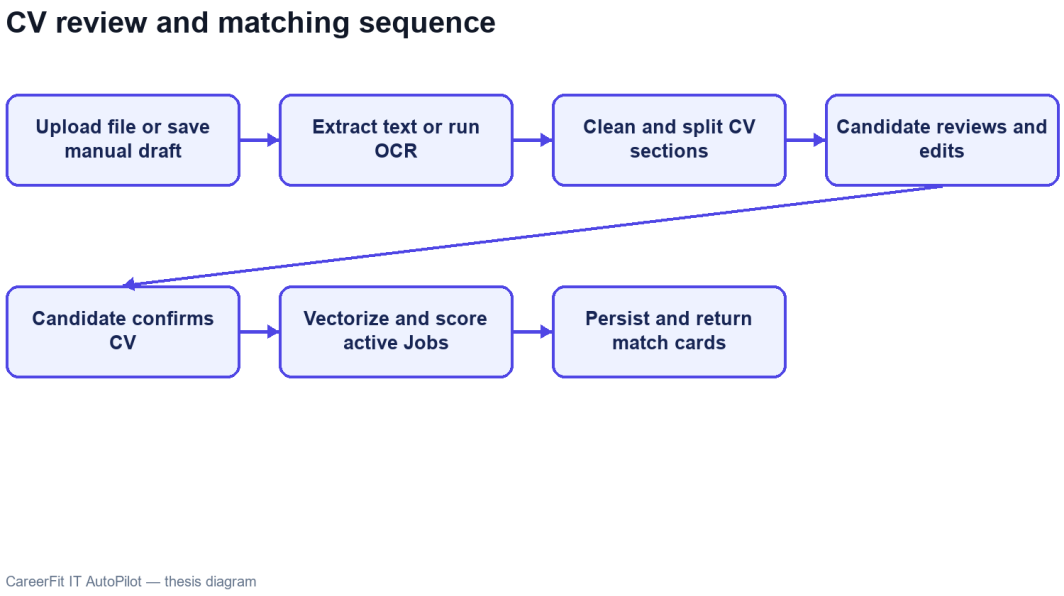


Figure 1.5. CV review, confirmation, and matching sequence

1.5.2 Candidate Searches, Applies, and Withdraws

Table 1.6. Use case - Candidate searches, applies, and withdraws

Field	**Description**
Use-case ID	UC-02
Primary actor	Guest or Candidate
Preconditions	Public search needs no account; application and personalized scoring require an authenticated Candidate.
Trigger	The user searches Jobs, opens details, or selects Apply/Withdraw.
Main flow	Search and filter public Jobs; open Job details; resolve the Candidate and selected/default CV; verify the Job; prevent duplicate Candidate-Job applications; create an application; list owned applications; withdraw when the current state permits.
Alternative/exception flows	Reject missing authentication, missing/default CV, invalid Job, duplicate application, unrelated ownership, or invalid withdrawal state.
Postconditions	A valid application is created or withdrawn, and the resulting state is returned to the owning Candidate.

1.5.3 Recruiter Creates a Job and Reviews Candidates

Table 1.7. Use case - Recruiter creates a Job and reviews candidates

Field	**Description**
Use-case ID	UC-03
Primary actor	Recruiter

Field	Description
Preconditions	The Recruiter account is active, authenticated, and has completed its company profile.
Trigger	The Recruiter saves a Job draft or opens an owned Job/Talent Pool workspace.
Main flow	Validate structured and free-text data; preview JD quality; save DRAFT; set deadline/urgent state; publish; vectorize after commit; review applicants and Potential CVs; bookmark, invite, or update application state.
Alternative/exception flows	Reject missing company data, invalid deadline/content, another Recruiter's Job, invalid invitation, or unsupported status transition; keep incomplete work as DRAFT.
Postconditions	The Job draft or published Job and permitted recruitment actions are persisted with ownership and audit evidence.

1.5.4 Candidate Feedback Changes Later Ranking

Table 1.8. Use case - Candidate feedback changes later ranking

Field	Description
Use-case ID	UC-04
Primary actor	Candidate
Preconditions	The Candidate owns the CV and Matching associated with the feedback.
Trigger	The Candidate submits GOOD_MATCH, POTENTIAL, BAD_MATCH, or NOT_INTERESTED feedback.
Main flow	Resolve the Matching and actor; verify ownership and role; upsert one current judgment; commit the transaction; start Rocchio learning after commit; mark affected Matchings for recomputation; rescore and clear the marker.
Alternative/exception flows	Reject another role, another Candidate's Matching, missing evidence, or invalid feedback; retain retry visibility when asynchronous recomputation fails.
Postconditions	The feedback and audit state are stored, and later rankings use the rebuilt learned vector.

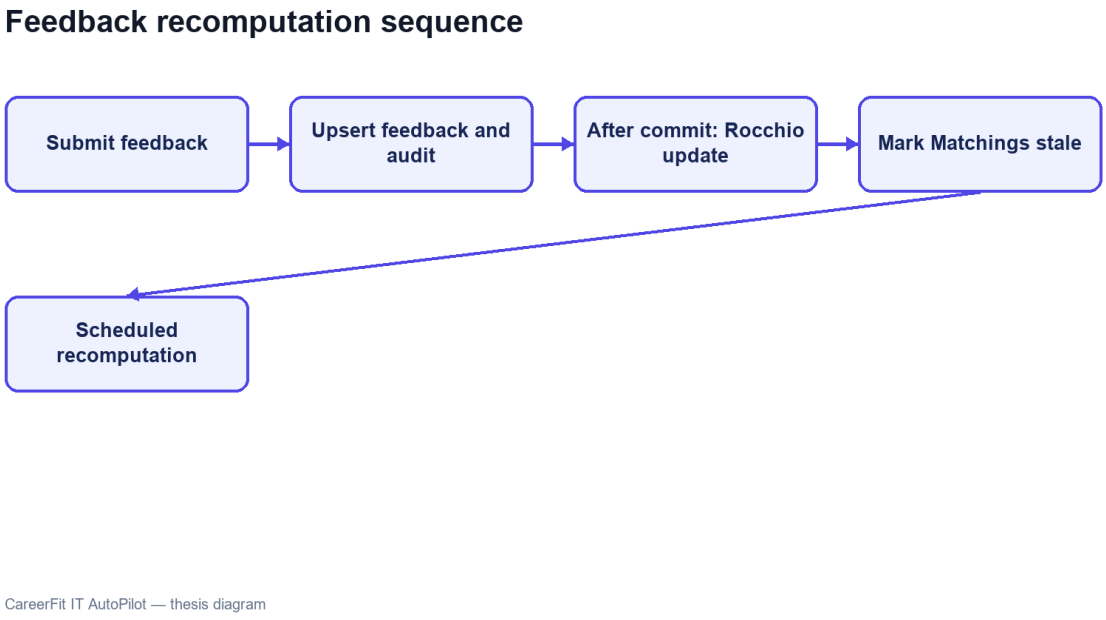


Figure 1.6. Feedback learning and recomputation sequence

1.5.5 AutoFit Executes a Policy-Governed Action

Table 1.9. Use case - AutoFit executes a policy-governed action

Field	Description
Use-case ID	UC-05
Primary actor	Candidate and background scheduler
Preconditions	An enabled owned policy, an eligible default CV, completed scoring, and an allowed action state exist.
Trigger	A scheduled scan or controlled run-now request evaluates the policy.
Main flow	Resolve the user, Candidate, CV, matches, thresholds, consent, quota, cooldown, quiet hours, time zone, and prior interactions; select eligible actions; create notifications or applications; write audit evidence.
Alternative/exception flows	Skip disabled, paused, ineligible, duplicate, quota-exceeded, cooldown, quiet-hour, stale, or invalid-state items; isolate per-item failures.
Postconditions	Only policy-allowed actions are recorded, and the remaining items stay unchanged for later evaluation.

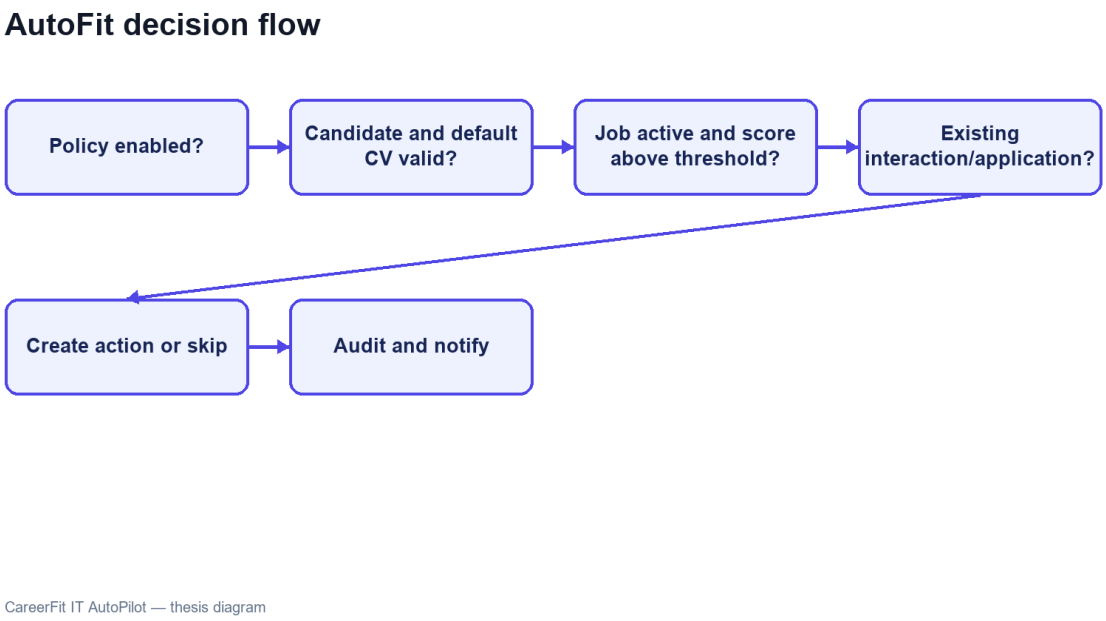


Figure 1.7. AutoFit decision flow

1.5.6 User Completes an Email Feedback Action

Table 1.10. Use case - User completes an email feedback action

Field	Description
Use-case ID	UC-06
Primary actor	Email recipient
Preconditions	A pending, unexpired EmailAction exists and only its SHA-256 token hash is stored.
Trigger	The recipient opens the link and deliberately confirms the action.
Main flow	GET hashes and validates the token and displays a confirmation page without changing recruitment data; POST repeats validation, performs the supported action, marks the token redeemed, and records the outcome.
Alternative/exception flows	Reject missing, expired, redeemed, malformed, or unsupported actions; mark expired pending records; do not execute state changes from GET.
Postconditions	The requested action is executed at most once under normal validation and remains auditable.

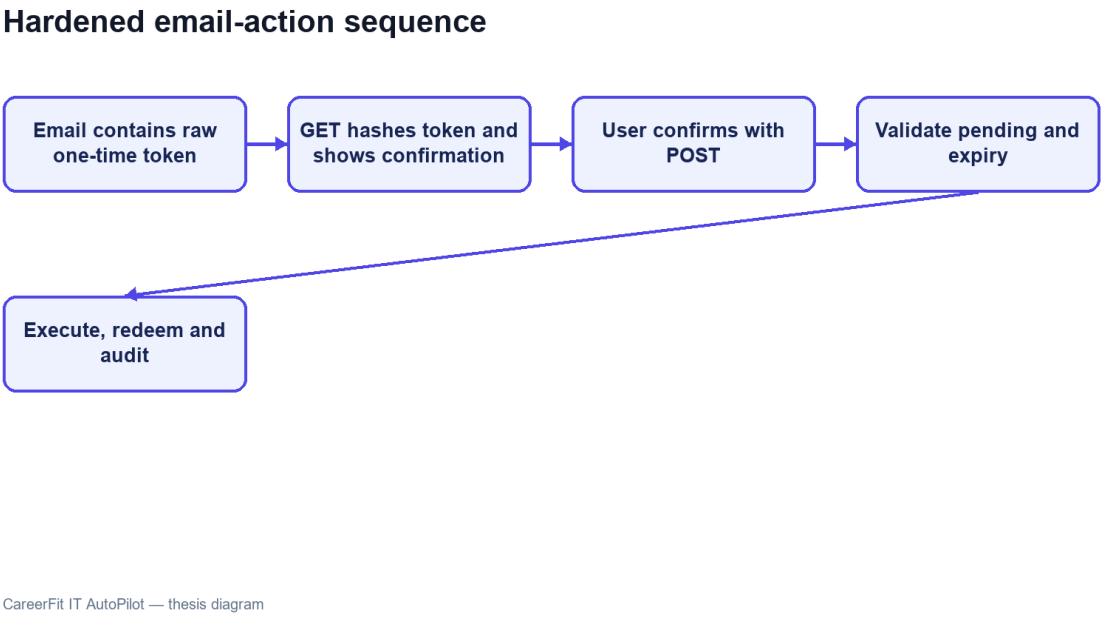


Figure 1.8. Secure email-action confirmation and redemption sequence

1.6 Chapter Summary

This chapter defined CareerFit's actors, functional and non-functional requirements, and six representative use cases. The requirements separate matching evidence, application state, user policy, reported-content state, automated actions, and audit records so later design and implementation decisions can be traced to expected behavior.

CHAPTER 2. THEORETICAL BACKGROUND AND SOLUTION DESIGN

2.1 Recruitment Information and CV–Job Description Matching

Recruitment matching is an information-retrieval and decision-support problem that compares different types of evidence about candidates and vacancies. A curriculum vitae usually describes education, work history, projects, technical skills, certifications, languages, and contact information. A job description lists responsibilities, required and preferred skills, seniority, location, employment conditions, and company context. Some data is structured, such as years of experience or location, while other data is free text and may use inconsistent terms. Therefore, exact database filters are useful for fixed conditions but are not enough to rank the overall fit between a candidate and a position.

The same information supports different user tasks. From the candidate perspective, the system retrieves and ranks jobs that fit a CV or desired profile. From the recruiter perspective, it ranks applicants or discovers potential candidates for a specific vacancy. These directions share document representations and similarity functions, but they are not identical business decisions. A relevant job recommendation does not imply that an application has been submitted, and a high candidate–job similarity does not imply that a recruiter should accept the candidate. The score is therefore evidence for prioritization rather than a final employment decision.

Recruitment data also has domain-specific ambiguity. A technology may be expressed by an abbreviation, a product name, or a broader skill family; job titles vary across companies; and the same term may have different importance at junior and senior levels. Open recruitment corpora are relatively uncommon because CVs contain personal information. The Djinni Recruitment Dataset illustrates both the scale of the matching problem and the value of anonymized, documented corpora, providing more than 150,000 jobs and 230,000 candidate profiles in English and Ukrainian [6]. CareerFit is narrower: it focuses on IT recruitment and uses explicit fields and normalized text so that matching behavior can be inspected within the project scope.

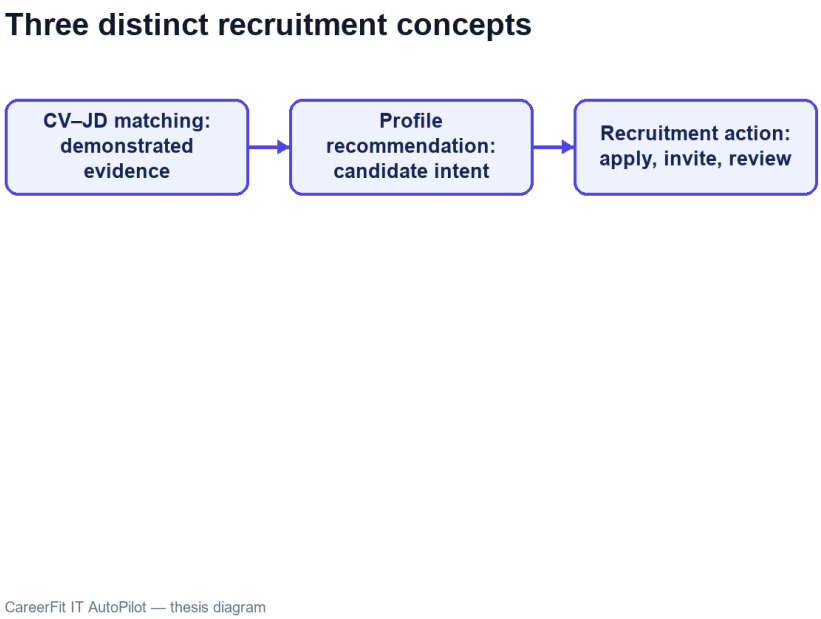


Figure 2.1. Distinction between matching, recommendation, and recruitment action

2.2 Text Representation and Preprocessing

2.2.1 Document Normalization

Before vectorization, text must be converted into a consistent form. A typical pipeline extracts machine-readable text, normalizes character encoding and letter case, separates or standardizes tokens, removes formatting noise, and may filter stop words or apply stemming or lemmatization. For recruitment documents, preprocessing must keep important technical tokens such as framework names, programming languages, versions, and abbreviations. Removing too much information can make a CV and a JD appear less related even when they share an important technology.

Bilingual data adds further complexity. Vietnamese contains diacritics and multi-syllable expressions separated by spaces, while English technical terminology frequently appears unchanged inside Vietnamese sentences. A practical pipeline must apply deterministic normalization to equivalent inputs and maintain a controlled vocabulary. It should also keep structured attributes—such as location, language, seniority, and salary mode—available for validation or filtering instead of forcing every business constraint into a single text vector.

Preprocessing quality directly affects every later score. Missing OCR text, malformed documents, extremely short descriptions, duplicated boilerplate, and contradictory structured fields can distort term frequencies. For this reason, validation belongs before scoring. Hard validation rejects inputs that cannot support

meaningful processing, while soft validation allows processing but records warnings or suggestions. This distinction prevents the ranking engine from silently assigning apparently precise scores to unusable data.

2.2.2 Bag-of-Words Representation

In a bag-of-words representation, a document is represented by the terms it contains and their weights; word order is not modeled directly. If the vocabulary contains |V| terms, each document is mapped to a vector in R^|V|. This representation is sparse because a particular CV or JD contains only a small portion of the complete vocabulary. Its main advantages are computational simplicity, reproducibility, and the ability to inspect which terms contribute to similarity. Its main limitation is lexical dependence: synonyms and related skills do not match unless normalization or an explicit mapping connects them.

2.3 TF-IDF and Cosine Similarity

2.3.1 Term Frequency and Inverse Document Frequency

The vector-space model represents documents and queries as weighted term vectors and ranks documents according to their geometric relationship [1]. TF-IDF is a standard weighting family for this model [2]. Term frequency (TF) reflects how strongly a term occurs in a particular document. Inverse document frequency (IDF) reduces the influence of terms that occur in many documents and increases the relative influence of terms that distinguish a smaller subset of the corpus.

Let tf(t,d) denote the frequency of term t in document d, let N be the number of documents in the reference corpus, and let df(t) be the number of documents containing t. A basic formulation is:

$$tfidf(t,d) = tf(t,d) \times log(N / df(t)).$$

Implementations often use logarithmic TF scaling, smoothed IDF, or vector normalization to handle repeated terms and unseen values. The exact formulation changes numeric scores, so a thesis must document the implemented formula rather than referring to TF-IDF as if it were a single universal function. A controlled or static reference corpus can improve score stability because the IDF of a term does not change whenever a user adds one document; however, it may become less representative as technologies and hiring terminology evolve.

2.3.2 Cosine Similarity

Cosine similarity compares the direction of two vectors rather than their unnormalized magnitude. For vectors x and y, it is defined as:

$$cos(x,y) = (x \cdot y) / (||x||_2 \, ||y||_2).$$

For non-negative TF-IDF vectors, cosine similarity usually lies between 0 and 1. A larger value indicates a greater proportion of shared weighted terms. Length normalization is valuable for CV–JD comparison because a long CV should not receive a higher score merely because it contains more words. TF-IDF-weighted cosine distance has also been used as an efficient and reproducible document-alignment method [3].

Cosine similarity remains a lexical measure. If a JD uses "container orchestration" while a CV only uses "Kubernetes," the vectors may not overlap unless the vocabulary or normalization layer establishes that relationship. Conversely, shared generic terms can increase similarity without proving that mandatory requirements are satisfied. CareerFit therefore treats similarity as one component of a broader workflow that also includes structured fields, validation, labels, reasons, user feedback, and policy conditions.

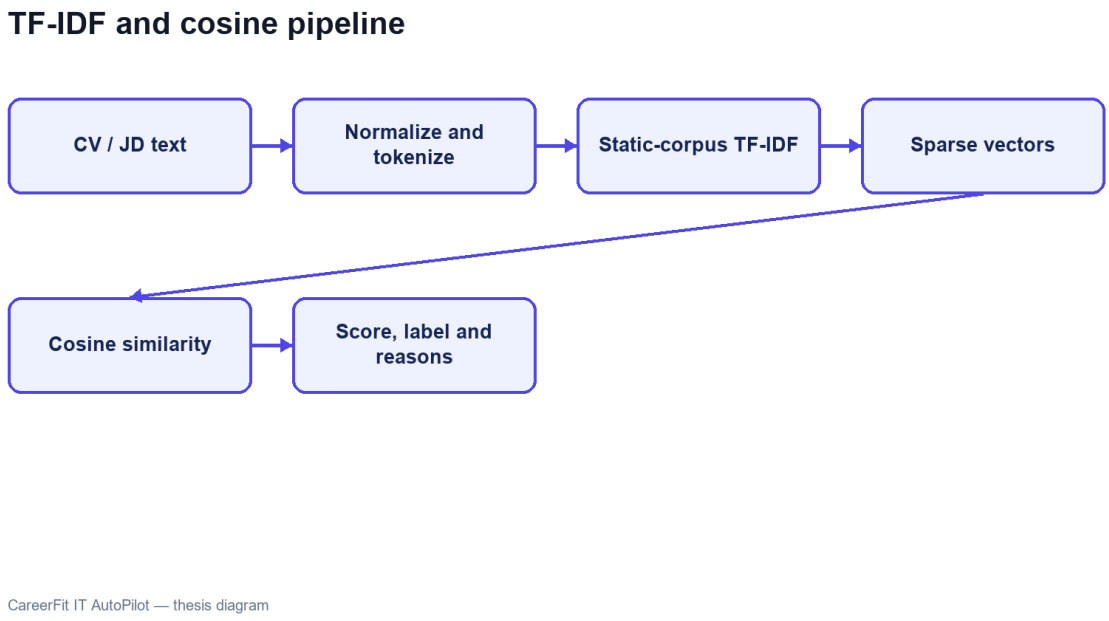


Figure 2.2. TF-IDF vectorization and cosine-similarity pipeline

2.4 Matching, Recommendation, and Ranking

Matching and recommendation can be formalized as ranking tasks. Given a query representation q and a candidate set D, the system computes a relevance score s(q,d) for each d □ D and orders the results by decreasing score. For candidate-to-job matching, q may be a CV vector and D the active job vectors. For recruiter-side discovery, q may be the selected JD and D the available candidate or CV vectors. For profile-based recommendation, q represents the candidate's desired role, skills, location, seniority, and other preferences rather than one uploaded CV.

This separation matters because a CV describes past experience, while a preference profile describes what the candidate wants. A candidate may have Java experience but look for a Go position, so CV-based matching and preference-based recommendation can produce different but valid lists. Keeping the workflows separate also makes evaluation clearer because the labels, candidate sets, and user goals of one task should not be assumed to fit the other task.

A raw similarity value is not automatically a calibrated probability of hiring success. Mapping it to a percentage or label is a presentation and business-rule decision, not a statistical guarantee. Thresholds such as Low, Medium, High, or Potential should be documented, tested at boundary values, and kept distinct from application status. Stable secondary sorting is also necessary when multiple items have equal scores so that repeated requests do not appear inconsistent.

Recent resume–job matching research increasingly uses dense encoders and contrastive learning. ConFit v2, for example, improves resume–job representations through hypothetical resumes and hard-negative mining [7]. Such approaches can capture semantic relations beyond exact terms, but they require training data, model governance, and an evaluation design appropriate to the domain. CareerFit intentionally begins with a transparent lexical baseline whose calculations can be reproduced and explained, while treating semantic or hybrid retrieval as future work rather than an implemented result.

2.5 Relevance Feedback and the Rocchio Method

2.5.1 Relevance Feedback

Relevance feedback uses judgments about retrieved items to modify a query or profile representation. Instead of assuming that the original text completely expresses the user's information need, the system observes which results are marked relevant or non-relevant. Explicit feedback is especially useful in recruitment because the importance of related skills may be known to the candidate or recruiter but omitted from the original CV, profile, or JD.

The classic Rocchio method updates a query vector by combining the original query with the centroids of relevant and non-relevant document vectors [4]. Let q_0 be the original vector, D_r the set of relevant feedback documents, and D_{nr} the set of non-relevant documents. A common form is:

$$q_m = \alpha q_0 + (\beta / |D_r|) \Sigma(d \in D_r)\, d - (\gamma / |D_{nr}|) \Sigma(d \in D_{nr})\, d.$$

The parameters α, β, and γ control retention of the original representation, attraction toward relevant examples, and movement away from non-relevant examples. They are hyperparameters rather than universal constants. Positive and

negative feedback may also be weighted differently because an explicit rejection can be ambiguous: a user may reject a job because of location, timing, or salary even when its technical content is relevant.

2.5.2 Feedback Semantics in CareerFit

CareerFit distinguishes feedback such as GOOD_MATCH, POTENTIAL, BAD_MATCH, and NOT_INTERESTED. These labels should not be collapsed without a documented mapping. GOOD_MATCH provides a strong positive relevance signal. POTENTIAL may represent partial or transferable fit rather than direct relevance. BAD_MATCH is a negative judgment about the match. NOT_INTERESTED can reflect personal intent and may be weaker evidence about technical similarity. The implementation chapter must state exactly how each event affects the learned vector and whether role or channel changes its weight.

A reliable update process keeps the original base vector and rebuilds the learned vector from the current feedback history. Updating the previously learned vector again and again can cause cumulative drift: the same recomputation may change the output even when there is no new feedback. In CareerFit, the same base vector, feedback set, and parameters should always produce the same learned vector. This repeatable behavior is important for retries, scheduled jobs, debugging, and audits.

Feedback learning should be evaluated by comparing rankings before and after a defined feedback event in a controlled setup. The experiment records the baseline ranking, adds feedback, rebuilds the representation, and measures the effect on separate holdout items. If the same positive item is used both as feedback and as the only proof of improvement, the result is circular. Chapter 4 therefore separates feedback examples from holdout evaluation and clearly labels synthetic scenarios.

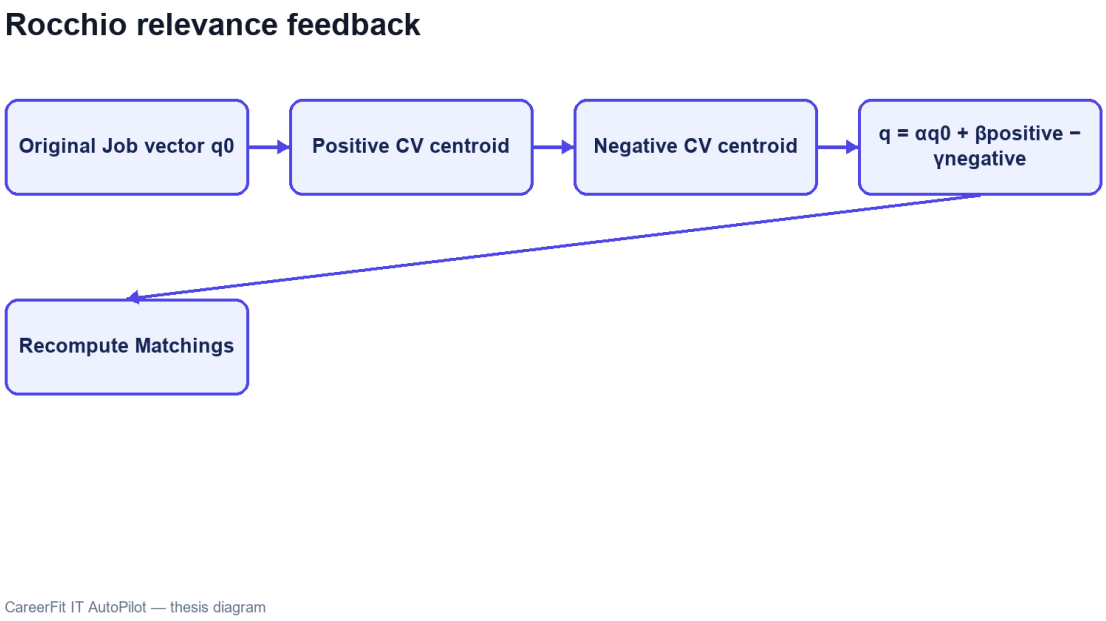


Figure 2.3. Rocchio relevance-feedback vector update

2.6 Evaluation Metrics for Ranked Results

Ranking quality cannot be described well by ordinary classification accuracy because users see an ordered list and usually inspect only the first few items. Evaluation therefore needs relevance judgments and metrics that consider the cutoff K, the positions of relevant results, and graded relevance when it is available.

2.6.1 Precision@K, Recall@K, and HitRate@K

Precision@K is the proportion of the top K results that are relevant. Recall@K is the proportion of all known relevant items that appear in the top K. HitRate@K records whether at least one relevant item appears in the first K positions. Precision emphasizes the usefulness of the visible list, recall emphasizes coverage, and HitRate provides a simple task-success indicator. The metrics answer different questions and should not be substituted for one another.

2.6.2 Mean Reciprocal Rank

For a query, reciprocal rank is 1/r, where r is the rank of the first relevant result; it is zero if no relevant result is retrieved. Mean Reciprocal Rank (MRR) averages this value across queries. MRR strongly rewards placing the first relevant result near the top but ignores the ordering of additional relevant items. It is therefore appropriate when finding one useful job or candidate is the primary objective, but it should be accompanied by a broader top-K metric when multiple relevant results matter.

2.6.3 Discounted Cumulative Gain

Discounted Cumulative Gain (DCG) supports graded relevance and discounts useful items that occur at lower ranks. Normalized DCG divides the observed DCG by the ideal ordering for the same relevance judgments, producing nDCG values that are comparable across queries. Järvelin and Kekäläinen introduced cumulated-gain measures to credit systems for ranking highly relevant documents before less relevant ones [5]. One common formulation is:

$$DCG@K = \Sigma(i=1..K)\ (2^{rel_i} - 1) / \log_2(i + 1), \quad nDCG@K = DCG@K / IDCG@K.$$

Metric values are meaningful only when the relevance labels, candidate set, cutoff K, and calculation method are documented. A large improvement in a synthetic scenario shows behavior only in that scenario; it does not prove hiring quality in production. Chapter 4 therefore reports the data source, baseline, holdout logic, repeated-run behavior, and threats to validity together with the metric values.

2.7 Human-in-the-Loop and Explainable Automation

2.7.1 Levels of Human Involvement

Human-in-the-Loop is not a single interface feature. Human involvement can occur during data validation, model configuration, review of recommendations, approval of actions, exception handling, and post-action feedback. NIST describes human–AI configurations as ranging from manual decision making through systems that provide an additional opinion or defer to an expert, to more autonomous operation [9]. The appropriate configuration depends on the consequences and failure modes of the task.

Recruitment decisions can strongly affect people, so a similarity engine should help users set priorities instead of acting as the final decision-maker. CareerFit separates input processing, decision support, actions, learning, and audits. The matching engine provides evidence; the AutoFit policy checks whether an action is allowed; users can review or confirm important actions; feedback changes later rankings; and audit records support investigation. This design does not remove bias, but it makes the control points and system state clearer.

2.7.2 Explainability and Auditability

Explainability describes why a result or action was produced. Auditability asks whether the inputs, rules, actor, channel, time, and outcome can be checked later. A list of shared skills can explain lexical similarity, but it does not prove that the whole model or decision is fair. In the same way, an audit log records what happened but does not prove that the policy was correct.

Research on algorithmic hiring warns that claims about reducing bias and improving performance must be checked against real practices, data, and organizational context [10]. Explanations can help users understand a result, but they should be based on the features that the system actually uses. Research on explainable recommendation also separates scoring from the user-facing reasons shown with a result [14]. CareerFit therefore uses match reasons and policy outcomes that can be checked instead of unsupported natural-language claims about candidate suitability.

The NIST AI Risk Management Framework groups AI risk work into governance, mapping, measurement, and management [9]. CareerFit is not a complete implementation of this framework. However, its design follows several practical ideas: state the scope and limitations, keep people responsible for important actions, measure behavior with defined evidence, and keep records for later review.

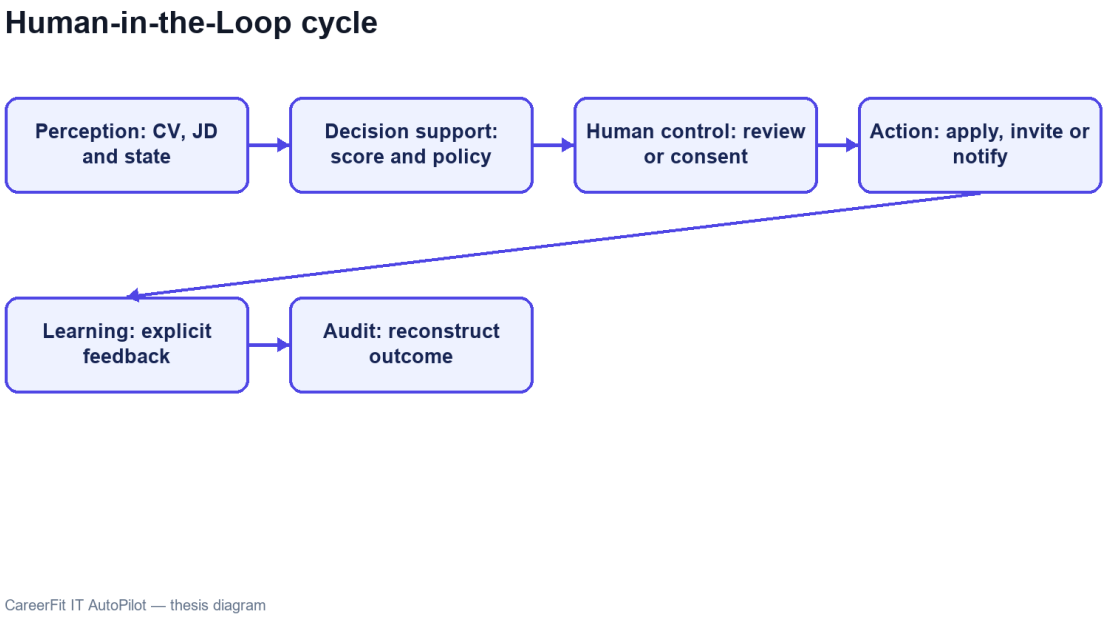


Figure 2.4. Human-in-the-Loop control cycle in CareerFit

2.8 Web Architecture, Security, and Audit Foundations

2.8.1 Client–Server and REST

CareerFit uses a web client and a server-side application connected through HTTP APIs. The client is responsible for presentation, navigation, local interaction state, and invoking authorized operations. The backend enforces authentication, authorization, validation, business rules, transactions, persistence, and background processing. This separation prevents browser-side presentation logic from becoming the authority for protected actions.

REST is an architectural style characterized by constraints including client–server separation, stateless interaction, cacheability where appropriate, a uniform interface, layered components, and optional code-on-demand [11]. A REST API models resources and transfers representations of their state. In practice, merely using JSON over HTTP does not guarantee that every REST constraint is satisfied; Chapter 3 therefore describes CareerFit as a REST-oriented API and documents its actual endpoint and state-transition behavior.

2.8.2 Authentication, Authorization, and Action Tokens

Authentication establishes the identity associated with a request, while authorization determines whether that identity may perform an operation. Role-Based Access Control assigns permissions according to roles such as Candidate, Recruiter, or Administrator, but ownership checks are still required; for example, one candidate must not access another candidate's private CV merely because both share the same role.

JSON Web Token is a compact, URL-safe format for transferring claims that can be signed or message-authentication-code protected and may include registered claims such as subject, audience, expiration time, issued-at time, and token identifier [12]. A valid signature alone is not sufficient: the application must validate the expected algorithm and relevant claims and must apply its own account and authorization rules. Short-lived access tokens and purpose-specific action tokens address different workflows and should not be treated as interchangeable credentials.

An actionable email link may be opened by the intended user, forwarded by mistake, or visited by a mail-security scanner. CareerFit uses a two-step flow to reduce this risk. The GET request checks the high-entropy token and shows a confirmation page without changing recruitment data. A deliberate POST request performs the action and marks the token as redeemed. Only the SHA-256 token hash is stored, and status and expiry checks prevent normal reuse. Production

deployment should still add rate limiting, origin controls, secret rotation, and mail-delivery monitoring.

2.8.3 Audit Logging

Audit logging records security-relevant and business-relevant events so that operations can be monitored and investigated. NIST guidance emphasizes a managed logging process that includes generation, transmission, storage, access, analysis, and disposal rather than treating logs as incidental text files [13]. For recruitment automation, useful fields include the actor, action, target, source channel, relevant policy or score snapshot, result, and timestamp. Sensitive CV content and credentials should not be copied into logs without a clear need.

An append-oriented audit history supports traceability, but database permissions, retention, integrity, and access control determine whether that history is trustworthy. Operational logs used for debugging and domain audit records used to explain a business action may have different schemas and retention requirements. Chapter 2 defines these responsibilities at the design level, and Chapter 3 describes the mechanisms actually implemented in CareerFit.

2.9 Related Work and Research Gap

Prior work can be grouped into lexical retrieval, learned resume–job representation, explainable recommendation, and human-centered governance. The classical vector-space and relevance-feedback literature provides transparent mathematical foundations [1], [4]. Contemporary resume–job matching methods such as ConFit v2 use trained dense representations and hard-negative strategies to improve semantic matching [7]. Real-world observational research comparing human and LLM resume ratings found only minor correlation between the two, indicating that automated and human judgments should not be treated as interchangeable [8]. Explainable recommendation research seeks to accompany scores with understandable, preferably grounded reasons [14].

CareerFit does not attempt to outperform these research systems on a shared benchmark. Its engineering gap is different: connecting an inspectable lexical baseline and explicit relevance feedback to an end-to-end recruitment workflow with role-based interfaces, policy gating, actionable email, and auditable state changes. The resulting contribution is system integration and controlled automation within an IT-focused academic prototype.

Table 2.1. Positioning of CareerFit against major approach categories

Approach	Strength	Limitation relative to CareerFit scope	CareerFit position
Lexical TF-IDF/cosine	Transparent, efficient, reproducible	Limited semantic equivalence	Implemented baseline with normalization, reasons, and validation
Dense resume–job encoders	Captures semantic relations	Requires training data, governance, and benchmark validation	Future hybrid/semantic extension; not claimed as implemented
General job portal	Strong search, publication, and application UX	May not expose scoring, feedback, or policy internals	Combines portal workflows with inspectable matching and automation
Fully automated decision pipeline	Reduces manual steps	Higher control, accountability, and error-propagation risk	Policy-gated HITL actions with confirmation and audit
Explainable recommender	Provides user-facing reasons	Explanation may be unfaithful if not grounded	Uses reasons tied to processed fields and lexical evidence

The comparison shows that CareerFit deliberately accepts the semantic limitations of a lexical baseline in exchange for inspectability and implementation control. Its research value must therefore be assessed through reproducible behavior, workflow integration, and clearly stated limitations rather than through unsupported claims of general AI superiority.

2.10 Theoretical Background Summary

This chapter presented the main ideas used by CareerFit. TF-IDF and cosine similarity provide an inspectable lexical baseline, while Rocchio uses explicit feedback to adjust ranking. Ranking metrics measure ordered results, and Human-in-the-Loop policies keep similarity evidence separate from recruitment actions. Chapters 1 and 2 turn these ideas into system requirements and design decisions.

2.11 System Architecture

CareerFit is implemented as a modular monolith. The frontend is a separate React application, while the backend is one Spring Boot deployable application organized into domain packages. This is not a microservice architecture: modules share one process, one primary PostgreSQL database, common security and exception handling, and direct in-process service calls. The modular monolith reduces deployment and distributed-transaction complexity while preserving boundaries that can support later extraction if scale or ownership requires it.

The presentation layer contains React pages, reusable components, route guards, API mapping, and query state. It calls the backend over HTTP and must not be trusted to enforce protected business rules. The API layer contains controllers and DTO validation. The service layer owns use-case orchestration, transactions, scoring, policy decisions, and ownership checks. Repositories map domain entities to PostgreSQL through Spring Data JPA. Cross-cutting configuration covers security, CORS, async execution, exception responses, scheduling, API documentation, and health/metrics endpoints.

The backend stores relational business state in PostgreSQL. JSONB is used for naturally variable collections or snapshots such as skills, extracted terms, vectors, reasons, benefits, policy metadata, and analytic distributions. CV binaries are stored through a storage service in a configured local path or Docker volume; the database stores file metadata and the processing result. SMTP is optional, and a no-operation mail implementation allows local workflows when outbound email is disabled.

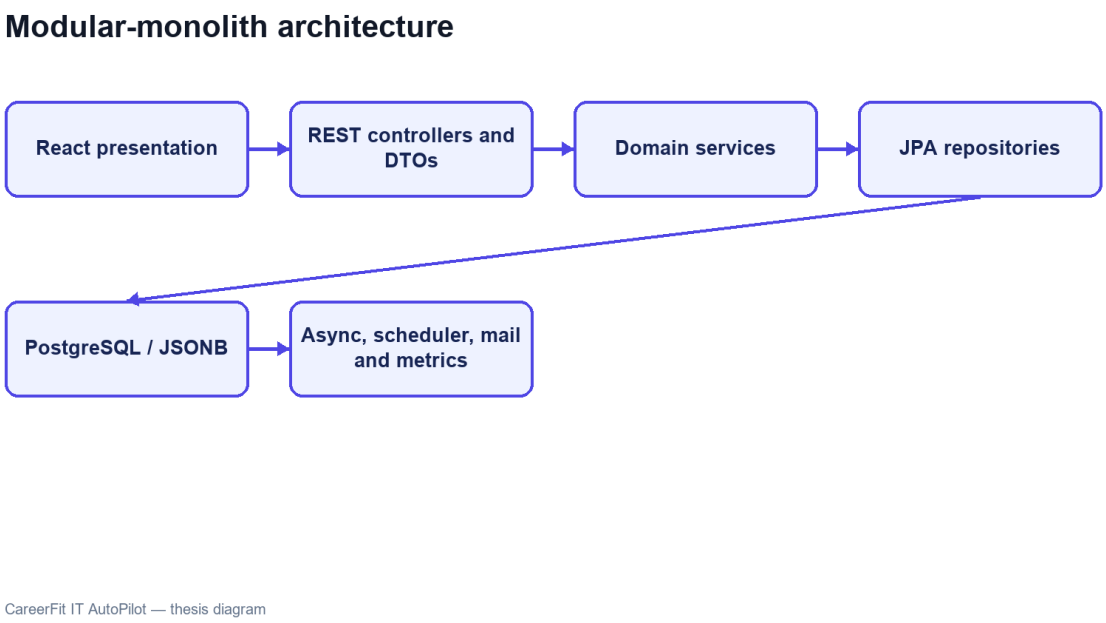


Figure 2.5. CareerFit container and component architecture

2.12 Module Design

Table 2.2. Backend modules and responsibilities

Module	Main responsibility	Representative services
Auth and Security	Password login, JWT validation, account resolution, role and route rules	AuthService, JwtService, security filters
Candidate, CV, and Skill	Profile, CV draft/review, storage, OCR, validation, and skill suggestions	CandidateProfileService, CvIngestionService, PdfExtractionService, SkillService
Job and Employer	Public catalogue, company onboarding, Job quality, draft/publish, urgent/deadline, and lifecycle	JobService, EmployerService, QualityValidationService
Matching and Recommendation	TF-IDF/cosine, reasons, Potential, batch matching, filters, ranking, and Job recommendation	ScoringService, SkillTransferService, MatchingService, RecommendationService
Application and Talent	Apply/withdraw, applicants, invitations, status changes, Talent Pool, and bookmarks	ApplicationService, RecruiterTalentService
Feedback and Automation	Rocchio update, recomputation, account policy, auto-apply, reminders, and scheduler	FeedbackService, RocchioService, AutoApplyService, AutomationScheduler
Notification	Policy guard, delivery log, SMTP/no-op mail, and signed email actions	NotificationPolicyGuard, NotificationEmailService, EmailActionService
Content Report	Job/CV reports, access checks, queue grouping, ban/dismiss, counters, and audit	ContentReportService, ContentReportController, AdminReportController
Analytics, Admin, and Audit	Live analytics, account/Job administration, monitoring, and audit persistence	Analytics services, administrative services, audit/common components

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use

dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

2.13 Data Design

2.13.1 Core Identity and Recruitment Data

user_account stores identity, role, activation, verification, language, and password hash. A Candidate owns a profile, multiple CVs, reviewable CV sections, extracted skills, portfolio links, and portfolio projects. A Recruiter owns an employer profile and Jobs. A Job stores its JD, structured recruitment fields, lifecycle status, report count, urgent flag, deadline, application count, source details, and vector data. A CV also stores its lifecycle status and pending report count.

matching is the unique association between one CV and one Job. It stores the direct score, label, separate Potential flag and explanation, reasons, and recomputation state. application links a Candidate and Job and may reference the CV and Matching used at that time. feedback stores a judgment about a Matching. recruiter_cv_bookmark keeps a Recruiter's saved Candidate/CV for an owned Job, while recommendation_interaction records events such as viewed, skipped, applied, saved, not interested, or show similar.

2.13.2 Automation, Communication, and Operational Data

automation_policy stores one policy per account, including enable/pause state, thresholds, quotas, cooldowns, quiet hours, category guards, and notification preferences. email_action stores expiring one-time actions using a token hash. content_report stores the reporter, JOB or CV target, reason, comment, pending/resolved state, resolving administrator, note, and timestamps. Notification delivery records support policy decisions, while audit_log records important user, moderation, and automated actions.

Table 2.3. Key integrity constraints

Constraint	**Purpose**
Unique normalized user email	Prevent duplicate identities that differ only by case
One role profile and one policy per account	Preserve ownership and account-level settings
Partial unique default CV per Candidate	Ensure at most one active default CV
Unique Matching per CV and Job	Prevent duplicate direct scores for the same pair
Unique Application per Candidate and Job	Prevent duplicate applications or invitations
Unique bookmark and feedback keys	Keep one bookmark/judgment for each defined actor-target relation

Constraint	Purpose
Unique pending report per reporter and target	Prevent duplicate unresolved abuse reports from the same account
Check constraints on roles, states, labels, report values, salary, and channels	Reject invalid domain values at the database boundary
Indexes for Jobs, ownership, ranking, audit, and report queue	Support catalogues, reminders, matching, and moderation queries

Flyway migrations are the authoritative schema history. Hibernate uses ddl-auto=validate, so the application checks entity-to-schema compatibility instead of silently generating a different database. Indexes support active Jobs, ownership, matching, applications, feedback, tokens, audit chronology, and the pending report queue. A partial unique index prevents one reporter from creating duplicate pending reports for the same target.

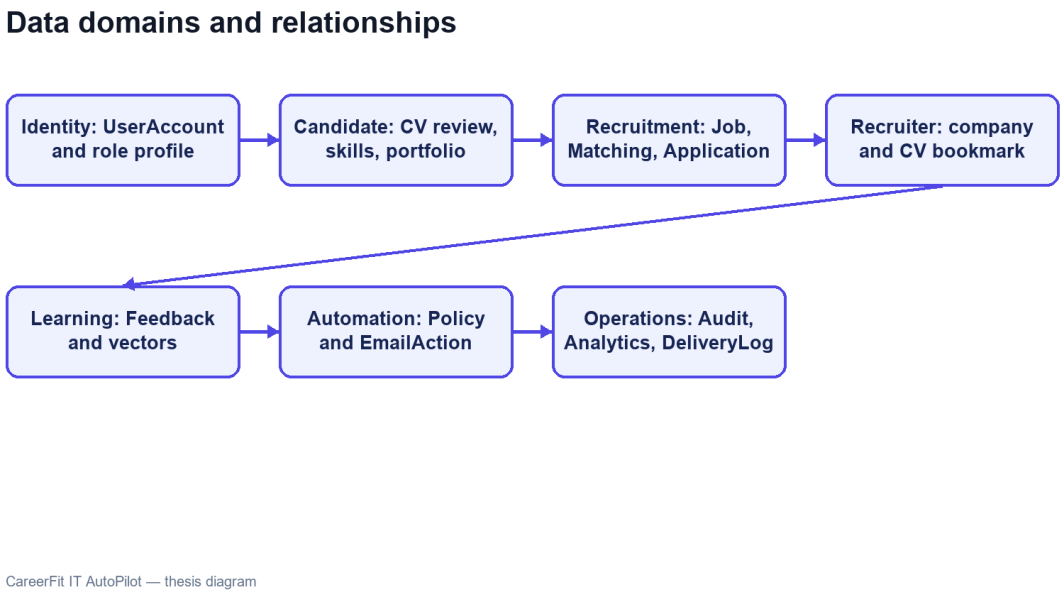


Figure 2.6. CareerFit logical entity relationships

2.14 Security, Failure, and Consistency Design

2.14.1 Authentication and Authorization

The backend is stateless at the HTTP session layer. A JWT authentication filter runs before username/password authentication processing, and a user-ID resolution filter runs after JWT validation. Public access is explicitly limited to authentication entry points, selected job/employer/analytics GET endpoints, health/metrics/API documentation, and email-action redemption. Candidate, Recruiter, and Administrator endpoint groups require corresponding roles. Automation requires authentication, with service logic responsible for policy ownership.

The security configuration disables CSRF because the application uses stateless bearer authentication, configures a CORS origin allow-list, denies framing, applies a content-security policy, and configures HSTS. These headers do not replace TLS termination in the deployed environment. Public Swagger and Prometheus access are acceptable for local development but should be restricted in a production deployment.

2.14.2 Failure Handling

CV processing represents long-running work with statuses rather than keeping a browser request open until every score completes. Extraction or validation failure records a failure state and reason. Matching tasks start after commit so background work cannot read uncommitted rows. Report moderation uses transactions and row locking to update the target, report counter, report resolutions, and audit state together. A banned Job is excluded from active flows, while a banned CV can no longer remain the default CV.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

2.14.3 Identified Design Risks

Table 2.4. Design risks and required treatment

Risk	**Current control**	**Remaining treatment**
Unauthorized cross-account access	URL roles and service ownership/visibility checks	Keep negative integration tests for identifier-based operations
Duplicate applications, Matchings, or reports	Service checks plus unique constraints/indexes	Convert database conflicts into stable API errors
False or abusive content reports	Fixed reasons, one pending report per reporter-target, and Administrator review	Add rate limits, evidence rules, moderator assignment, and abuse monitoring
Incorrect moderation action	Case detail, explicit Ban/Dismiss, transactions, and audit events	Add appeal, restoration, and dual-review policy for production
Replayed email action	Hashed token, pending/redeemed state, expiry, and POST execution	Retain replay, expiry, scanner, and rate-limit tests

Risk	Current control	Remaining treatment
Scheduler/configuration drift	Scheduler timing reads application properties	Maintain configuration and schedule-boundary tests
Sensitive data in logs	Structured audit metadata and DTO boundaries	Review retention, redaction, access, and exported evidence
Lexical scoring bias or omission	Reasons, validation, feedback, and HITL	Evaluate representative data; never treat score as hiring probability
Local file-storage loss or exposure	Configured path and Docker volume	Define backup, encryption, malware scanning, and retention

2.15 Deployment Architecture

The development deployment uses Docker Compose for PostgreSQL and optionally the backend. PostgreSQL 16 exposes container port 5432 as host port 5433 by default. A backend process running directly on the host connects to localhost:5433; the backend container connects through the Compose network to postgres:5432. Confusing these two addresses is a common configuration failure.

The optional backend container exposes port 8080, depends on PostgreSQL health, and mounts a named volume for CV storage. The React/Vite frontend is normally run separately on port 5173 during development and calls the backend API. Environment variables override database credentials, JWT secret, application base URL, CORS origins, mail settings, OCR executable and languages, and storage path. Tesseract with Vietnamese and English language data is included in the backend container path described by the project Docker configuration.

Actuator exposes health and Prometheus metrics according to the current security configuration. PostgreSQL health checks gate backend container startup. This architecture is appropriate for reproducible local development and demonstration, but it is not by itself a production topology. A production design would additionally require managed secrets, TLS termination, restricted management endpoints, database backup and recovery, protected object storage, centralized logs, monitoring alerts, and explicit scaling and retention policies.

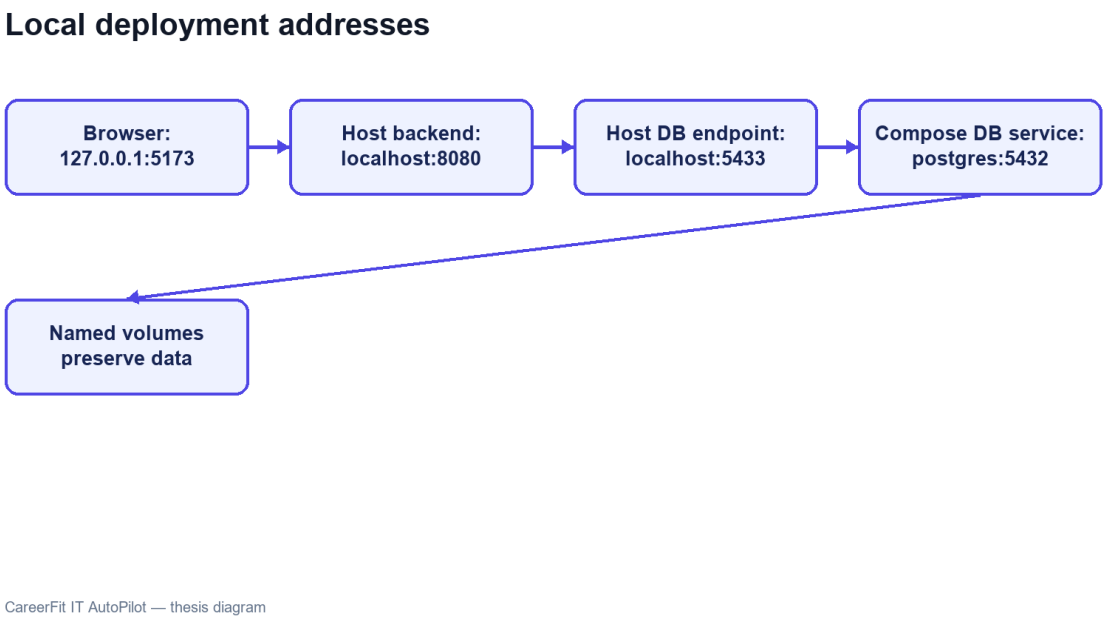


Figure 2.7. Local and containerized deployment topology

2.16 Chapter Summary

This chapter connected the theoretical foundations to the CareerFit solution design. TF-IDF, cosine similarity, Rocchio feedback, ranking metrics, and Human-in-the-Loop principles explain the core decision-support behavior. The architecture, modules, data model, security boundaries, consistency rules, and deployment topology define how those ideas are separated into implementable components.

CHAPTER 3. SYSTEM IMPLEMENTATION

3.1 Implementation Overview

CareerFit is implemented as two application projects and a shared runtime environment. The backend is a Java 21 application based on Spring Boot 3.2.5. The frontend is a React 18 single-page application written in TypeScript and built with Vite. PostgreSQL is the transactional database, Flyway owns schema migration, and Docker Compose provides the reproducible local database and optional backend container. The implementation follows the modular-monolith design established in Chapter 2: domain packages share one backend process while retaining controllers, services, repositories, entities, and DTOs for each functional area.

Table 3.1. Main implementation technologies

Layer	Technology	Implementation purpose
Backend language/runtime	Java 21	Domain logic, API, background processing, and integrations
Application framework	Spring Boot 3.2.5	Dependency injection, web MVC, configuration, scheduling, and application lifecycle
Security	Spring Security and JJWT 0.12.5	Stateless bearer authentication, role authorization, and JWT handling
Persistence	Spring Data JPA, PostgreSQL 16, Flyway	Entity persistence, queries, constraints, indexes, and versioned migrations
Document processing	PDFBox 3.0.2, Apache POI, Tesseract CLI	PDF/DOCX extraction and OCR fallback for images or scanned PDFs
API documentation	springdoc-openapi 2.5.0	OpenAPI documents and Swagger UI
Monitoring	Spring Actuator, Micrometer Prometheus registry	Health and HTTP/application metrics
Frontend	React 18.3, TypeScript 5.9, React Router 7, React Query 5	Role-based pages, typed API integration, navigation, and server-state queries
Visualization and UI	Recharts and Lucide React	Charts and interface icons

Layer	Technology	Implementation purpose
Build and test	Maven, npm/Vite, JUnit, Testcontainers, Playwright	Compilation, automated backend tests, integration environment, and browser E2E tests

The backend package root is com.careerfit.backend. Domain packages include auth, candidate, cv, skill, job, employer, matching, recommendation, application, feedback, automation, notification, analytics, report, admin, audit, and settings. Shared responses, exceptions, text utilities, validation, security filters, and configuration are placed in common or configuration packages.

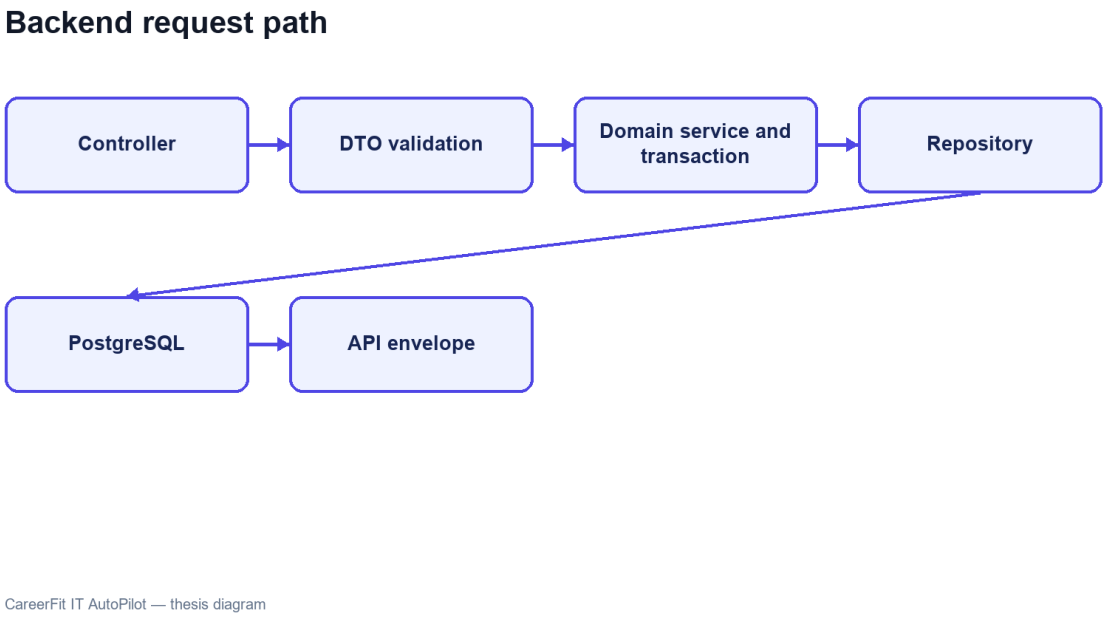


Figure 3.1. Backend module structure and request flow

3.2 Authentication and Authorization Implementation

3.2.1 Login and JWT Processing

AuthController exposes registration, email/password login, and current-account operations. AuthService checks account state and credentials and delegates JWT creation to JwtService. Passwords are stored with BCryptPasswordEncoder. A successful login returns an access token and a user DTO with the ID, email, full name, role, and verification state. Passwordless login was removed; signed one-time tokens remain only for explicit email actions.

Each authenticated request passes through JwtAuthenticationFilter, a OncePerRequestFilter. The filter requires the Bearer prefix when an Authorization header is present, validates token structure and signature, extracts subject and role, rejects roles outside Candidate, Recruiter, and Administrator, and reloads the

account from the database. Reloading prevents a cryptographically valid token from continuing to authorize a deleted or suspended account. The filter then creates a Spring Security authentication object with the corresponding ROLE_* authority.

UserIdResolutionFilter runs after JWT authentication. It resolves the persisted account ID from the authenticated email and stores the UUID as the userId request attribute. Controllers can therefore pass a stable identifier to services without accepting a user ID supplied by the browser. The implementation currently performs a user lookup in both filters; this is straightforward but creates duplicate database reads that could be consolidated later.

3.2.2 URL Rules and Ownership Checks

SecurityConfig uses a stateless session policy. Public access is limited to selected authentication operations, public Job/employer/analytics GET routes, similar-job retrieval, health/metrics/OpenAPI routes, and email-action redemption. Candidate and Recruiter report endpoints require authentication and are checked again by role and ownership in ContentReportService. Administrative report queue, detail, ban, and dismiss endpoints require the Administrator role.

Role checks are necessary but insufficient for resources addressed by UUID. ApplicationService, for example, verifies that a requested CV belongs to the authenticated Candidate and that a Recruiter owns the Job before listing applicants, changing status, or inviting a candidate. The same pattern is required across identifier-based services. Errors are returned through shared security and application error writers so that 401 unauthenticated and 403 forbidden responses remain distinguishable.

The configuration disables CSRF because bearer tokens are used instead of a server-side browser session. CORS origins are loaded from application configuration. Frame denial, Content Security Policy, and HSTS headers are configured. HSTS is effective only when the application is served through HTTPS, so the local HTTP runtime does not itself demonstrate transport security.

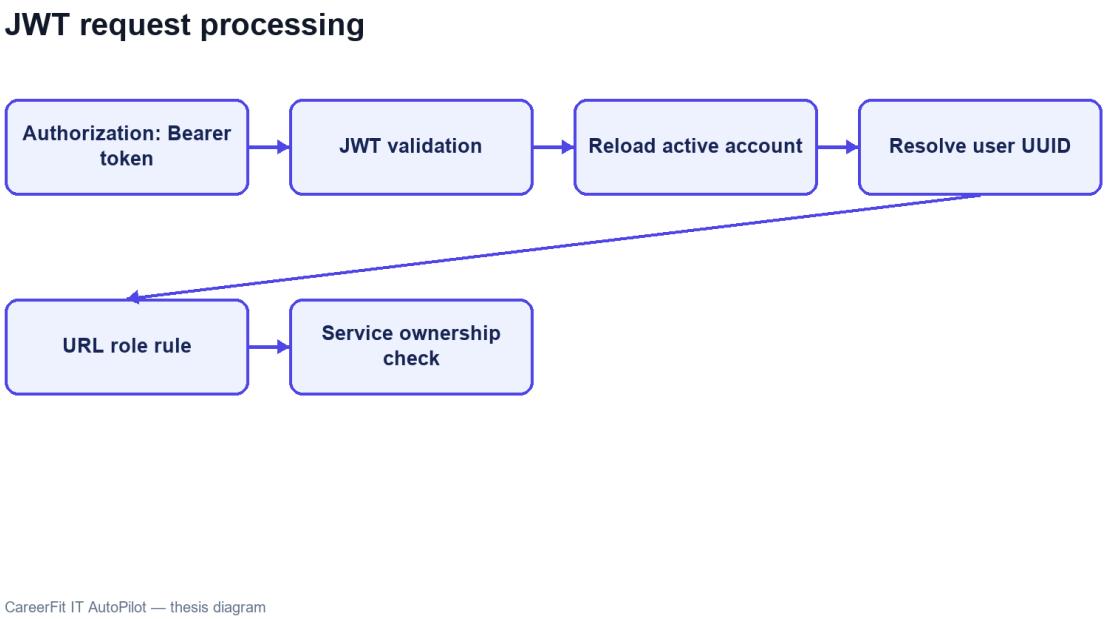


Figure 3.2. JWT authentication and authorization boundaries

3.3 CV Ingestion and Validation

3.3.1 Upload and Manual Creation

CvController provides upload, manual creation, manual drafts, CV listing, details, processing status, review read/update, review confirmation, default selection, and deletion. CvIngestionService coordinates the workflow. Uploaded content is extracted and converted into reviewable sections. Manual entry can be saved as a DRAFT. The Candidate can correct sections and confirm them before vectorization and Job scoring start.

Table 3.2. CV processing states

State	Meaning	Typical transition
DRAFT	Manual content is saved but not submitted for scoring	Draft edit → review/confirm
UPLOADED	Uploaded metadata and source are accepted	Upload → validation/extraction
VALIDATING	File, content, and extraction checks are running	Valid source → review; hard error → failed
REVIEW_REQUIRED	Extracted/manual sections are ready for Candidate review	Edit sections → confirm
PROCESSING	Confirmed content is normalized and vectorized	Vector saved → asynchronous scoring

State	Meaning	Typical transition
SCORING_DONE	The confirmed CV vector is available for matching	Use cards, feedback, applications, and AutoFit
FAILED	Processing cannot continue	Failure reason is stored and audited
BANNED	An Administrator acted on a report case	Removed as default and excluded from normal use

3.3.2 File Validation and Extraction

PdfExtractionService accepts PDF, PNG, JPG/JPEG, and DOCX. It checks extension, MIME type, and leading magic bytes. PDFBox extracts embedded PDF text, Apache POI extracts DOCX text, and images use OCR. Scanned pages are preprocessed before Tesseract runs, and OCR cleanup removes common noise before the text is split into review sections. The default OCR languages are vie+eng, with configured page and time limits.

Text shorter than 50 characters after extraction and cleanup is a hard failure. Text from 50 to 199 characters is accepted with a sparse-content warning. Encrypted PDFs are rejected, image dimensions are limited, and temporary OCR files are deleted. The review step also lets the Candidate fix extraction mistakes before the CV is used for scoring. Production use would still need malware scanning and stronger isolation.

QualityValidationService produces structured signals for questionable CV or JD content, such as missing sections, seniority/experience conflicts, salary/content conflicts, and invalid deadlines. Blocking errors are separated from warnings so users can fix the data instead of losing their work.

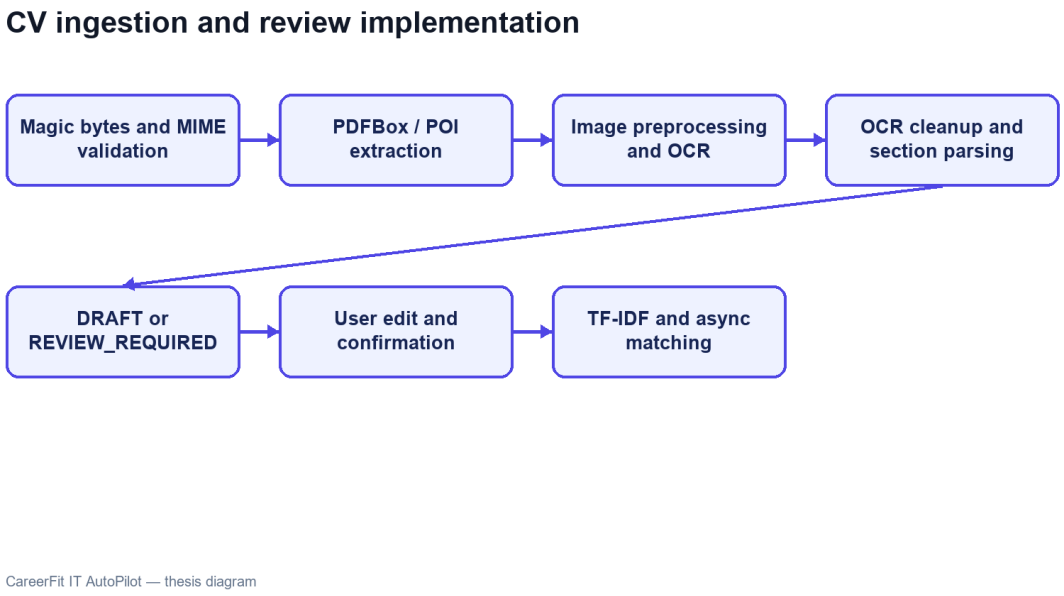


Figure 3.3. CV ingestion, review, and confirmation pipeline

3.4 Text Normalization and TF-IDF Implementation

3.4.1 Text Normalization

TextNormalizationService removes HTML tags, replaces characters outside its English/Vietnamese alphanumeric pattern, lowercases text, splits on whitespace, removes tokens shorter than two characters, and filters a language-specific stop-word set. Language detection is a lightweight heuristic: text containing more than five Vietnamese diacritic characters is classified as Vietnamese; otherwise it is treated as English.

This implementation is deterministic and easy to inspect, but it is not a linguistic word segmenter. Vietnamese multi-syllable skills can be split into independent tokens, and punctuation-containing technology names such as C++, C#, .NET, or Node.js may lose distinguishing characters. These limitations are important when interpreting matching errors and motivate a future domain-aware tokenizer or controlled alias dictionary.

3.4.2 Static Corpus and Vector Construction

TfIdfService builds its IDF map once at application startup from a static seed corpus of representative IT terms. The corpus groups programming languages, frameworks, databases, cloud and DevOps technologies, architecture, testing, security, data/ML, collaboration, seniority, roles, and common Vietnamese IT words. A static corpus prevents scores from shifting whenever a user uploads one CV or adds one Job.

For a token list, term frequency is the token count divided by the total number of tokens. The implemented smoothed IDF is:

$$idf(t) = log(1 + N / (1 + df(t))).$$

An unknown term receives log(1 + N), which treats it as rare and informative. The TF-IDF vector is persisted as JSON and reused during scoring. Cosine similarity iterates over the smaller vector for the dot product, computes both Euclidean magnitudes, and returns zero when either vector is empty.

The unknown-term policy has a practical trade-off. It preserves project-specific technology terms absent from the seed corpus, but a rare misspelling can receive the same high IDF treatment. Input validation, alias normalization, corpus versioning, and evaluation on realistic data are therefore necessary before treating the score as stable beyond the academic environment.

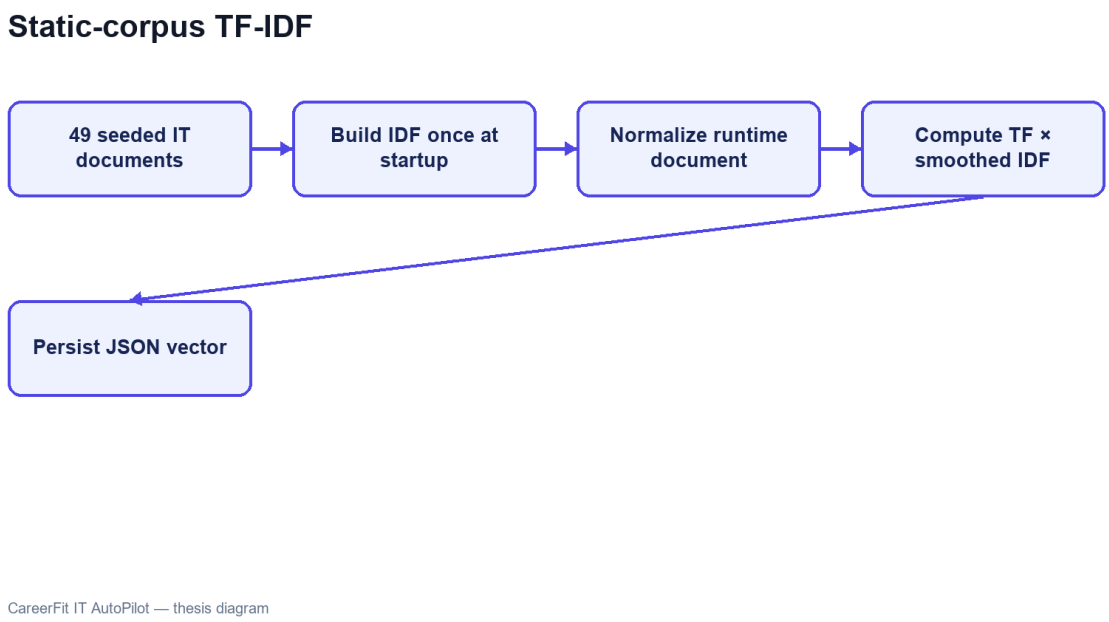


Figure 3.4. Seed-corpus initialization and TF-IDF construction

3.5 Matching and Recommendation Implementation

3.5.1 Scoring Service

ScoringService loads the CV vector from extractedTermsJson. For the Job, it prefers learnedProfileVectorJson when a non-empty learned vector exists; otherwise it uses the original tfidfVectorJson. Cosine similarity becomes the raw score, which is multiplied by 100 and rounded to two decimal places. The raw score is stored with six decimal places.

Table 3.3. Implemented score interpretation

Condition	Stored/displayed result

Condition	Stored/displayed result
Direct score below 70	LOW label
Direct score from 70 to below 90	MEDIUM label
Direct score at least 90	HIGH label; Potential is not added
Potential score at least 62 with skill compatibility ≥0.50 and family compatibility ≥0.55	Potential guard continues
Core target and transferable career evidence exist, with no severe seniority gap	isPotential=true and an explanation is stored
Direct score below 20 and skill compatibility below 0.70	Potential is rejected as weak evidence

The direct-match label uses the configured medium boundary (70) and high boundary (90), producing LOW, MEDIUM, or HIGH. Potential is a separate boolean and does not replace that label. The current Potential model is loaded from a versioned JSON knowledge base and is evaluated separately from cosine similarity.

Direct-match reasons use the highest-weight Job terms that also appear in the CV, with the Job domain added when available. Potential uses normalized aliases, direct skill-transfer links, role-family transfer, shared foundations, and seniority compatibility. Its explanation can list transferable skills and shared foundations. This remains a rule-based explanation, not a learned claim that the Candidate will succeed.

3.5.2 Matching Orchestration and Persistence

After the Candidate confirms the CV review, vectorization is committed and AfterCommitExecutor submits MatchingService.scoreAllJobsForCv. The service reloads the CV, scores eligible active Jobs, and creates or updates the unique Matching row. Reasons and Potential metadata are stored as JSON. The same post-commit boundary is used when a published Job is scored against existing confirmed CVs.

A Recruiter first completes a company profile, then can save a Job as DRAFT, check JD quality, and publish it with the required deadline. Published Jobs can be marked urgent and are scored after commit. MatchingQueryService supports Candidate and Recruiter views with server-side filters, pagination, sorting, label/Potential filters, and clear empty-state metadata. Portfolio details remain protected by the Candidate's visibility setting and application state.

3.5.3 Recommendation Service

RecommendationService serves personalized Job results and similar-job retrieval. Candidate recommendations use profile/preferences and available matching information, while similar jobs can be retrieved publicly. The API and UI intentionally keep recommendations separate from the persisted application workflow. A recommendation can be viewed, skipped, or used to start an application without becoming an application automatically unless an enabled AutoFit policy separately authorizes that action.

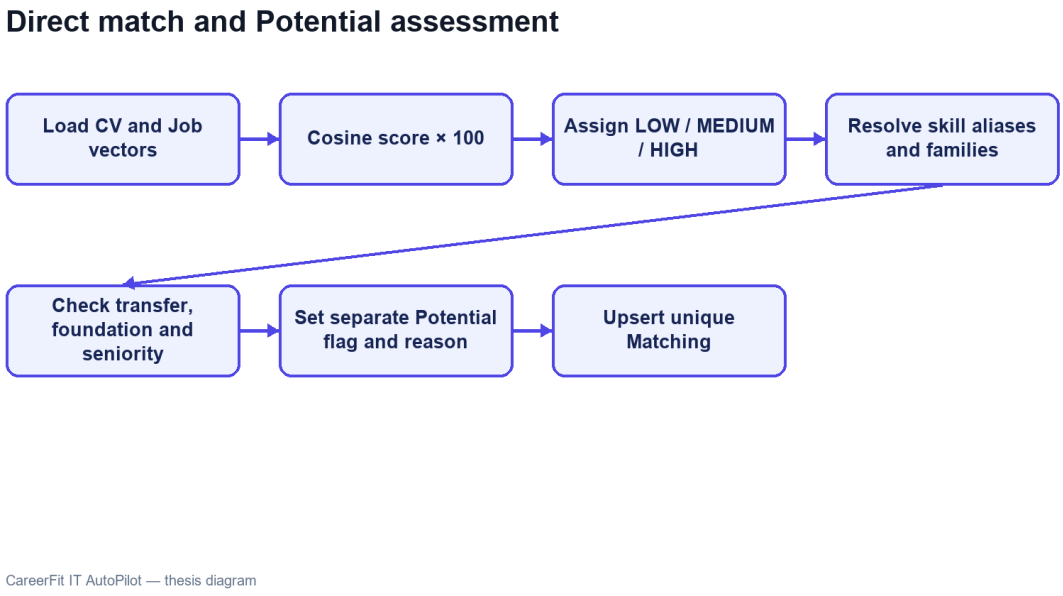


Figure 3.5. Direct score and Potential assessment flow

3.6 Feedback Learning with Rocchio

FeedbackService.submitFeedback resolves the Matching and authenticated Candidate, verifies that the Candidate owns the associated CV, and then upserts feedback by (matching_id, actor_id). Concurrent uniqueness conflicts are resolved by reloading and updating the existing row. Each event stores actor role, feedback type, and source channel and creates an audit entry. NOT_INTERESTED is stored but does not trigger Rocchio because it can represent preference rather than technical relevance. GOOD_MATCH, POTENTIAL, and BAD_MATCH trigger post-commit Job-vector learning.

RocchioService locks the Job for update and always loads the original Job TF-IDF vector as q. Positive examples are CV vectors attached to GOOD_MATCH or POTENTIAL feedback; negative examples come from BAD_MATCH. The service computes term-wise centroids and applies:

$$q\_new = 1.0q + 0.75\ positive\_centroid - 0.15\ negative\_centroid.$$

Negative weights are removed rather than persisted. The learned vector is stored in learnedProfileVectorJson. Every Matching associated with the Job is then marked needsRecompute=true. AutomationScheduler.recomputeStaleMatchings rescans these rows every 30 minutes and clears the marker only after successful scoring.

Using the original base vector and the complete feedback history makes recomputation repeatable for the same data and parameters. FeedbackService starts learning after the transaction commits, so Rocchio reads the saved feedback instead of racing the transaction that created it. Job locking prevents two updates from writing separate learned vectors at the same time, and integration tests cover this post-commit behavior.

Table 3.4. Feedback handling

Feedback	Persisted	Rocchio classification	Immediate learning trigger
GOOD_MATCH	Yes	Positive	Yes
POTENTIAL	Yes	Positive	Yes
BAD_MATCH	Yes	Negative	Yes
NOT_INTERESTED	Yes	Neither	No

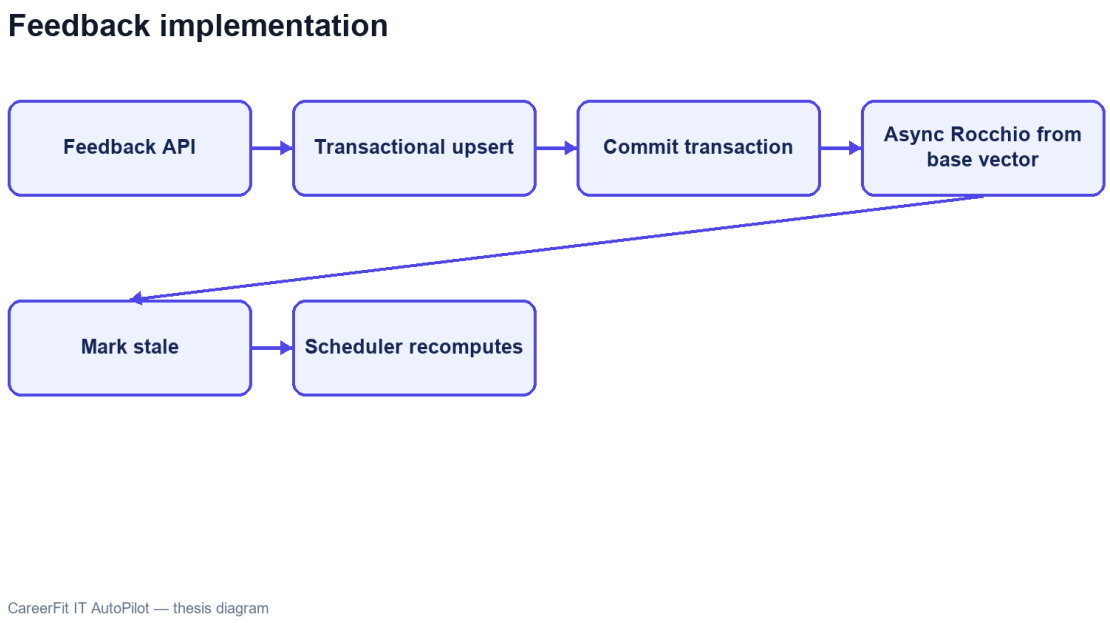


Figure 3.6. Feedback processing and post-commit learning

3.7 Application and Recruiter Workflow

ApplicationService.submit resolves the authenticated Candidate, requires an active Job before its application deadline, rejects an existing Candidate–Job application, and uses the requested or default confirmed CV. If a Matching exists, it is attached to keep the score context. The service saves an audit event, updates the Job application count through the database rule, and requests Candidate and Recruiter notifications.

A Candidate views owned applications through a paginated API and can filter them by status. A Recruiter can list applicants only for an owned Job, update valid states, invite a Candidate, and report a visible CV in the context of an owned Job. The Talent Pool adds Job-based Candidate discovery, a Potential-only view, and CV bookmarks. A Candidate can report an active Job from the Job interface.

Response DTOs combine application, Candidate, CV, Matching, Job, and bookmark details without exposing persistence entities. Portfolio fields are returned only when the Candidate has enabled visibility and the application state permits it. Server-side pagination, sorting, and filters keep larger Job and Candidate catalogues manageable.

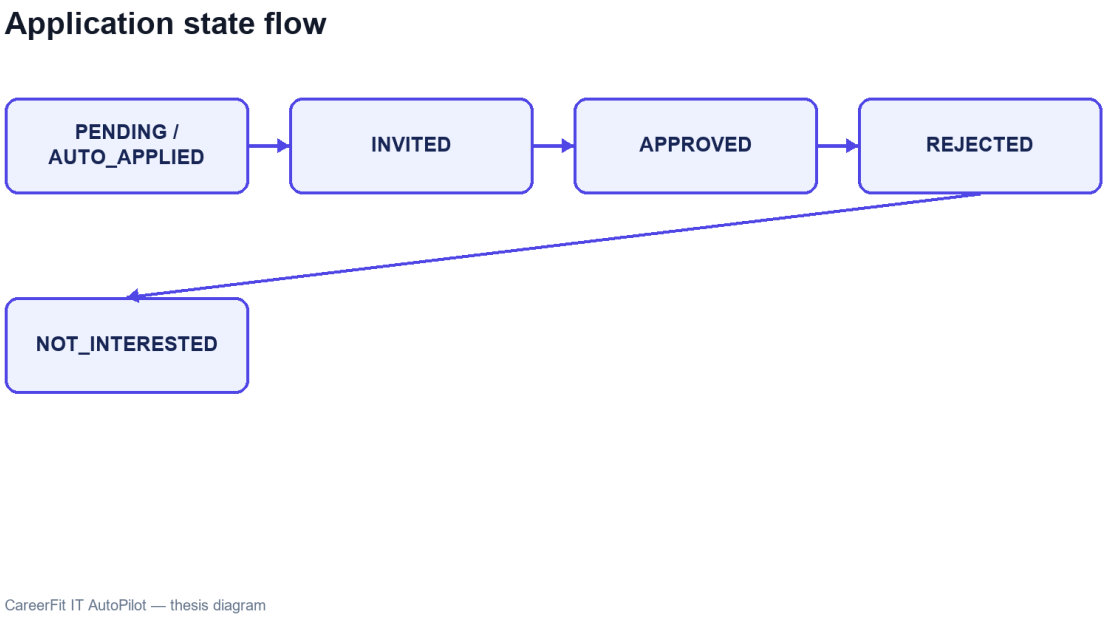


Figure 3.7. Application and invitation state transitions

3.8 AutoFit and Background Processing

3.8.1 Async Executor and Scheduler

AsyncConfig creates a bounded ThreadPoolTaskExecutor. AfterCommitExecutor submits CV and Job matching only after the related transaction commits. Mail sending and Rocchio learning can also leave the request thread, while persisted CV, Matching, email-action, and delivery states make completion and failures observable.

AutomationScheduler coordinates matching recomputation, digests, high-match checks, auto-apply, token cleanup, and deadline-related reminders using configured schedules. Per-item error handling prevents one failed record from stopping a full scan. Account policies add pause/resume, thresholds, quotas, cooldowns, quiet hours, and category guards before eligible actions or notifications are executed.

3.8.2 Auto-Apply

AutoApplyService.runForPolicy resolves the policy owner, Candidate, and confirmed default CV. It checks policy enablement and pause state, eligible categories, active Job state, deadline, score threshold, previous interactions, quota, and cooldown before creating an automatic application. Database uniqueness remains the final concurrency guard. Every created or skipped outcome can be audited and followed by a notification decision.

Auto-apply and notification delivery use related but separate checks. Application creation requires consent and application-policy conditions, while email delivery also checks timing, quota, cooldown, category, and deduplication. Keeping the two decisions separate prevents a notification setting from silently becoming permission to submit an application.

3.8.3 Notification Guard and Delivery

NotificationPolicyGuard evaluates whether a message is allowed and stores sent, skipped, or failed results in separate transactions. It supports deduplication, quota, cooldown, quiet hours, category guards, and deadline reminders. SMTP is used when enabled, while NoOpMailService keeps local development usable without claiming that real email delivery was tested.

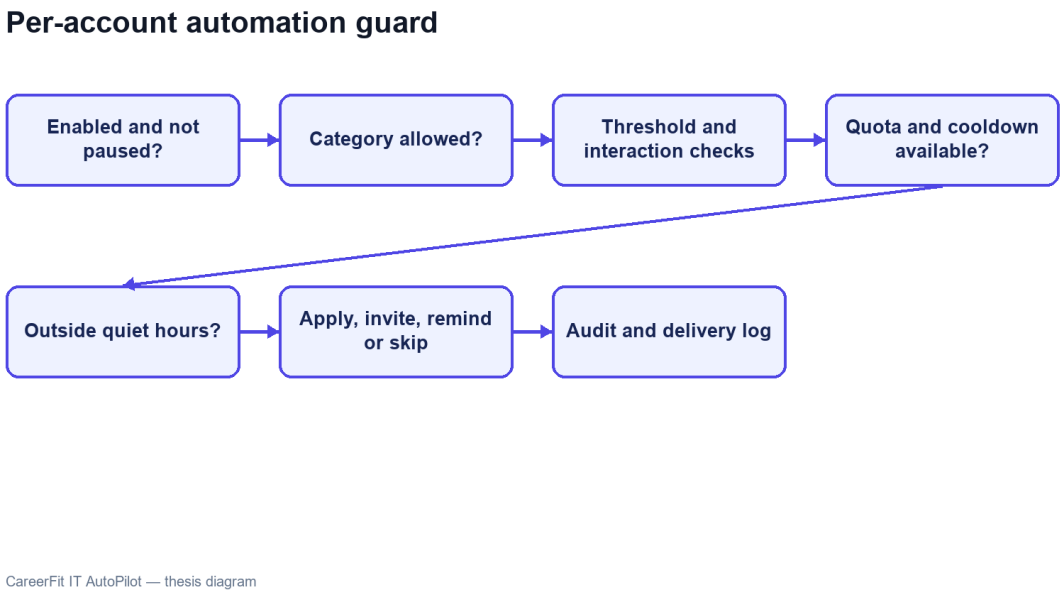


Figure 3.8. Per-account AutoFit policy guard

3.9 Email Action and Audit Implementation

EmailActionService generates a 32-character token derived from UUID text for each supported action and stores it with recipient, optional Matching, type, status, and expiry. Match email actions include GOOD_MATCH, POTENTIAL, NOT_INTERESTED, and view links. Digest messages create actions for listed Matches and unsubscribe intent. Tokens remain valid for 72 hours.

EmailActionController exposes a public confirmation and redemption flow because possession of the high-entropy token is the credential. The raw token is sent to the user but only its SHA-256 hash is persisted. GET validates the token and renders a non-mutating confirmation page; POST performs the supported action, records redemption, and rejects unknown, expired, or already-used records.

This implementation removes the earlier state-changing GET and raw-token persistence findings. The remaining production concerns are rate limiting, deployment-specific link origin, secret rotation, mail-delivery monitoring, explicit handling of expired links, and protected audit review.

Audit records are written directly through AuditLogRepository in major services. Entries can contain actor type/ID, action, target, result, source channel, and JSON metadata. Examples include CV failure, application submission, invitation, feedback, auto-apply, JOB_REPORTED, CV_REPORTED, report dismissal, and banning a Job or CV. Direct repository calls are simple, but a centralized audit facade could better standardize metadata, redaction, and failure behavior.

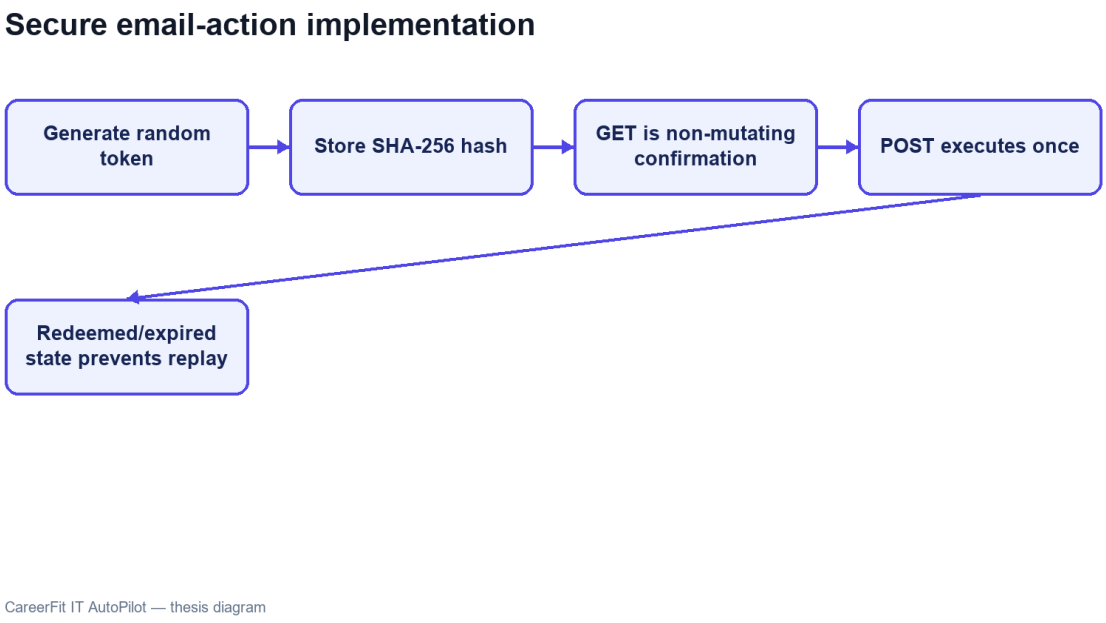


Figure 3.9. Hashed email-action token and confirm-then-POST flow

3.10 Persistence and API Implementation

JPA entities map the domain tables, and Spring Data repositories provide pagination, sorting, locking, and explicit update operations. Flyway migrations V1–V25 create and update the current schema. V25 adds content_report, report queue/target indexes, pending report counters, a unique pending-report rule, and the BANNED status for Job and CV. Hibernate uses ddl-auto=validate.

Database constraints are treated as correctness boundaries rather than only documentation. Unique indexes protect email identity, default CV, Matching, Application, imported Job hash, and duplicate pending reports by the same reporter and target. Check constraints restrict roles, labels, lifecycle states, report target/reason/status, source channels, and salary modes. Version columns and explicit row locks help protect mutable core entities.

Controllers return shared API envelopes and DTOs instead of exposing entities. Validation annotations reject malformed requests near the API boundary, while service exceptions represent not found, forbidden, bad request, and conflict cases. OpenAPI is generated through springdoc. Public list endpoints and authenticated workspaces use pagination or bounded top-result queries to avoid unbounded payloads.

Table 3.5. Representative API-to-service mapping

API operation	Controller/service responsibility	Persistence effect

API operation	Controller/service responsibility	Persistence effect
POST /api/cv/upload; GET/PATCH /api/cv/{id}/review	Extract content and expose/edit review sections	CV, review data, quality signals, audit
POST /api/cv/{id}/review/confirm	Confirm owned CV and start matching	CV vector, Matchings, audit
GET /api/matches/me/cards; POST /api/matches/{id}/feedback	Query Candidate cards and submit feedback	Feedback, audit, learned Job vector, stale flags
POST /api/jobs/drafts; POST /api/jobs/{id}/publish	Validate company/JD/deadline and manage owned Job	Job, vectors, Matchings, audit
PUT /api/recruiter/talent/.../bookmark	Validate Job ownership and save Talent Pool choice	Recruiter CV bookmark
PATCH /api/automation/policy	Update the account's automation policy	Automation policy and version
POST /api/reports/jobs/{id}; POST /api/reports/cvs/{id}	Validate role, target state, Job ownership/CV visibility, reason, and duplicate pending report	ContentReport, pending report count, audit
GET /api/admin/reports; POST .../{type}/{id}/ban or /dismiss	Load report cases and resolve all pending reports for a target	Target status/counter, report resolutions, audit

3.11 Frontend Integration

App.tsx defines public, Candidate, Recruiter, and Administrator routes. Protected routes check the loaded account role, while session restoration calls /api/auth/me and clears invalid state. Candidate pages include Job reporting. Recruiter applicant and Talent Pool views can report a visible CV with the owned Job ID. The Administrator workspace includes a Report moderation tab for the pending queue, case detail, ban, and dismiss actions.

Frontend/src/lib/api.ts centralizes HTTP access. It reads VITE_API_BASE_URL with /api as the default, attaches the bearer token from sessionStorage, selects JSON headers except for multipart FormData, unwraps the shared response envelope, and throws a normalized error for unsuccessful responses. DTO mapping functions convert backend shapes and labels into UI

models. React Query hooks manage loading, refetching, caching, polling, and error state for server data.

The frontend stores the access token and account summary in sessionStorage, limiting persistence to the current browser tab. This still exposes the token to any successful cross-site scripting attack. The Content Security Policy reduces some exposure, but production hardening should evaluate short-lived tokens, refresh rotation, and HttpOnly secure-cookie alternatives according to the deployed architecture.

API-driven pages do not replace failed responses with mock Job data. The current frontend uses server-side filters, pagination, sorting, CV review, recruiter onboarding, Job draft/publish, urgent and deadline controls, Talent Pool actions, AutoFit settings, and analytics. Reporting modals use fixed reasons and an optional comment; badges show pending report state, and the admin moderation tab groups pending reports by target.

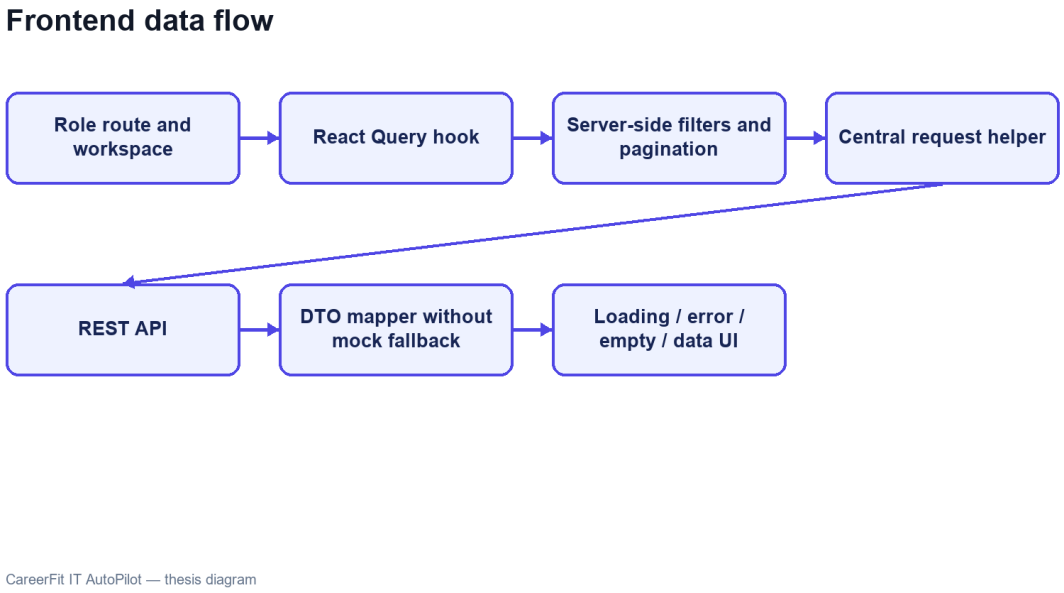
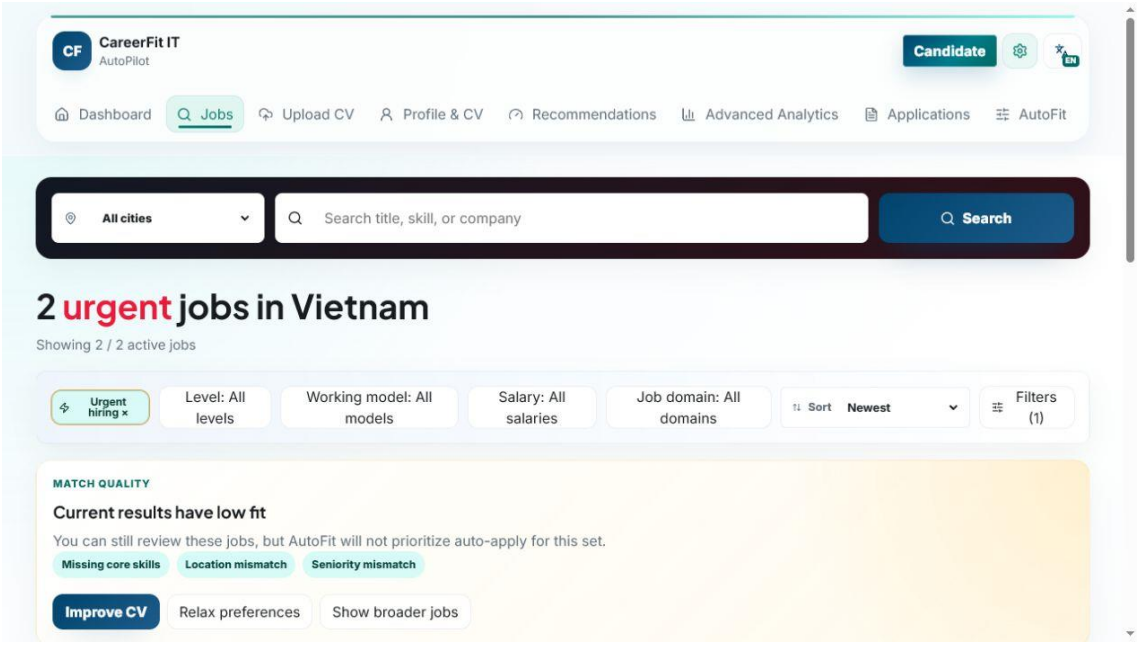
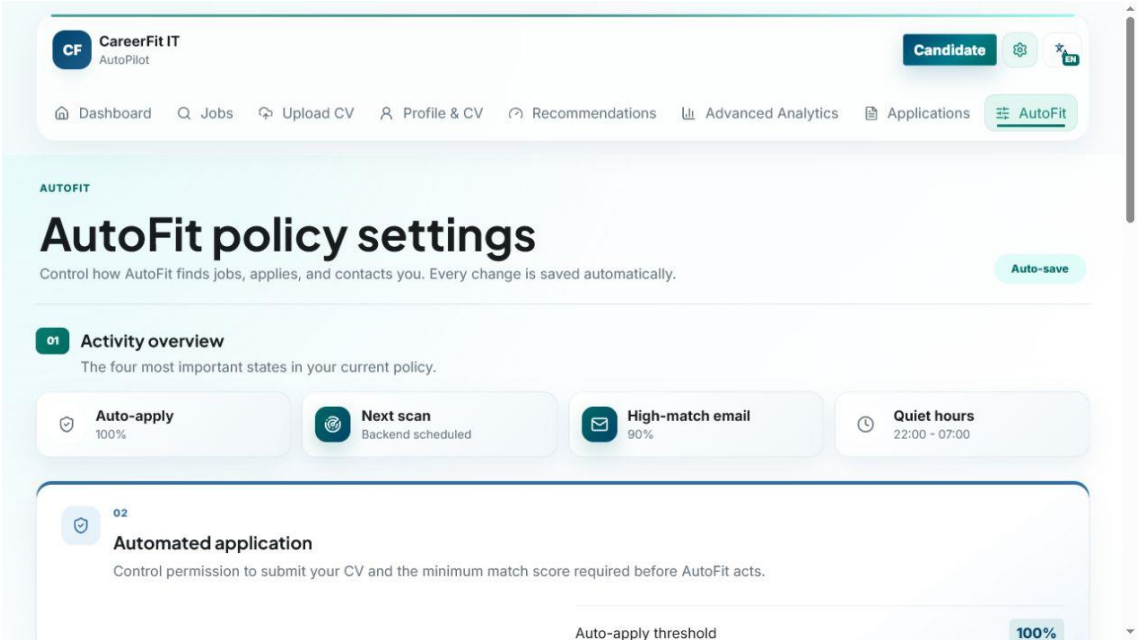


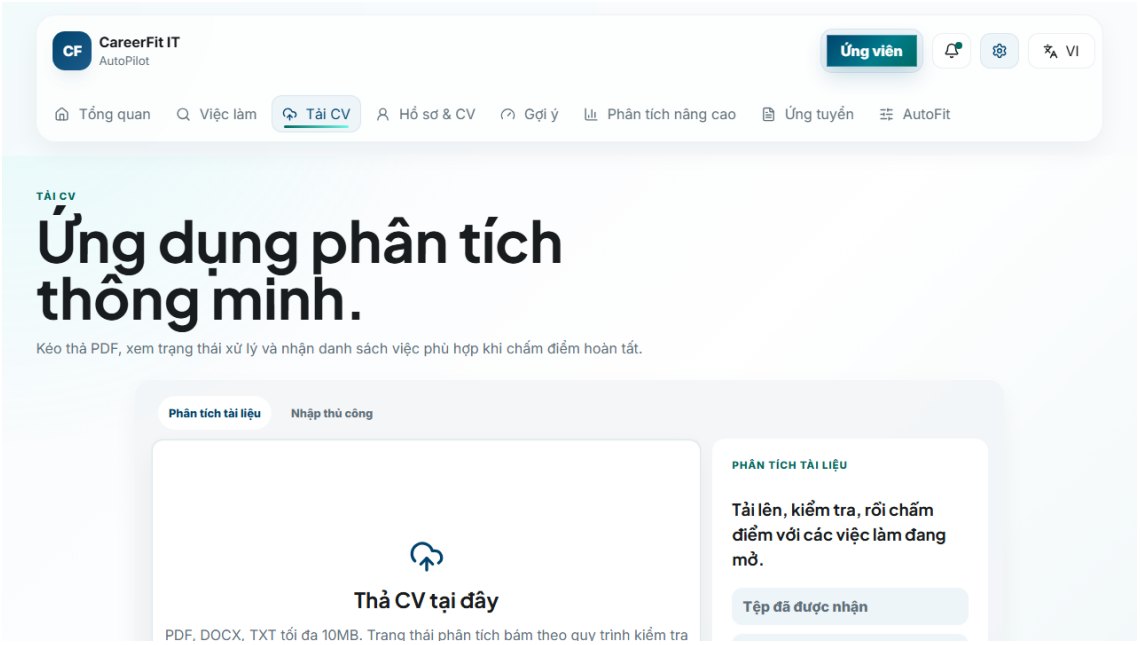
Figure 3.10. Frontend routes, server-side catalogue, and API data flow



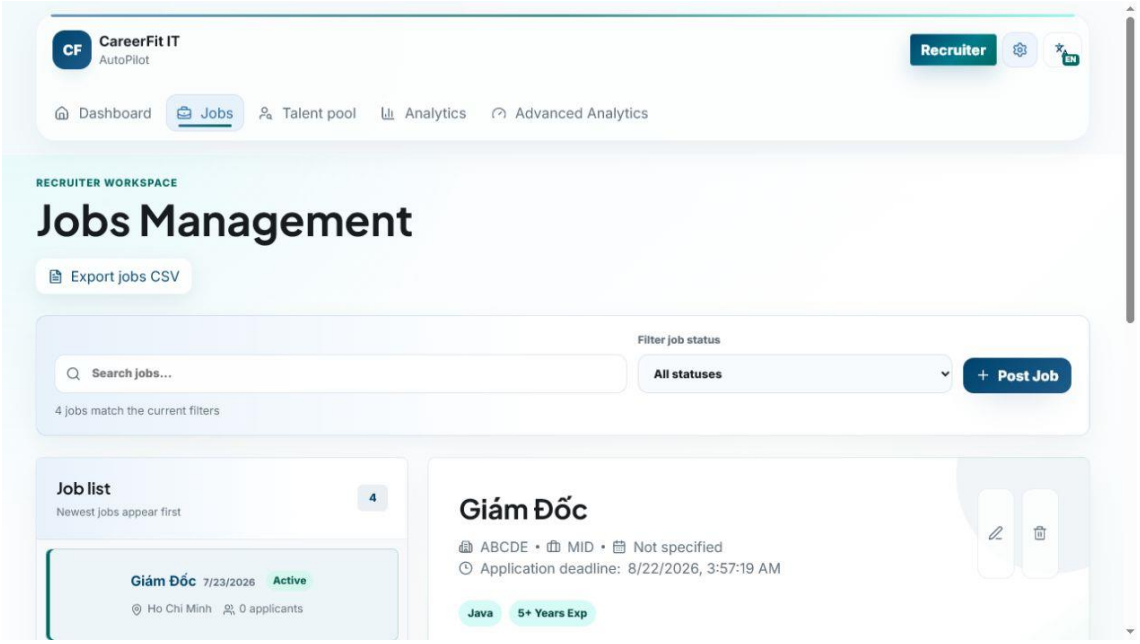
Screen 3.1. Candidate urgent-job catalogue



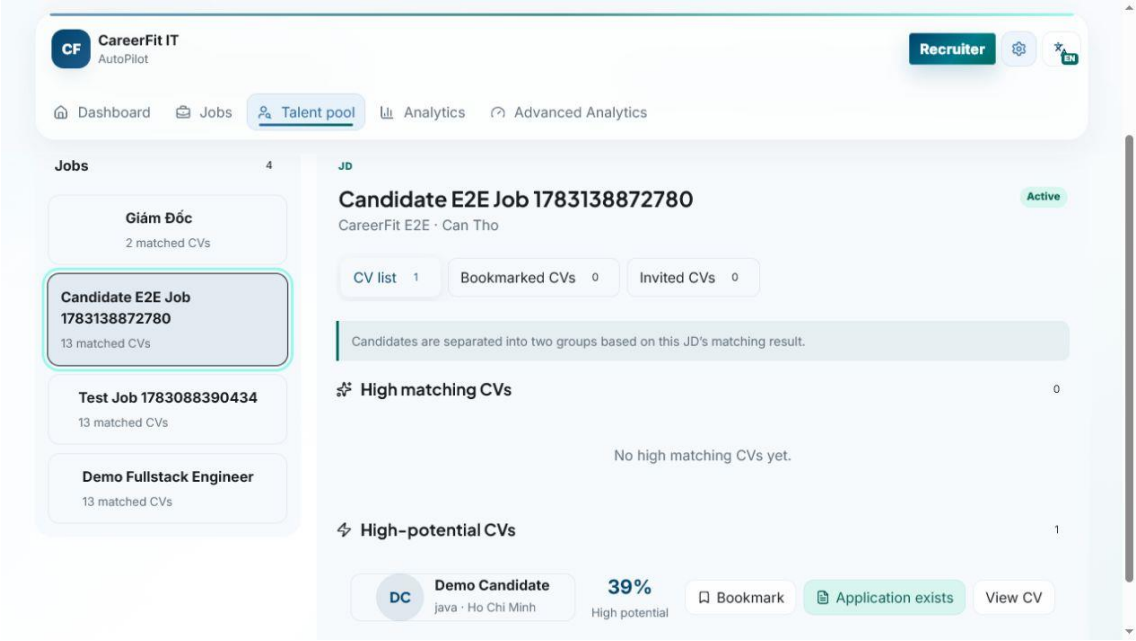
Screen 3.2. Candidate AutoFit policy settings



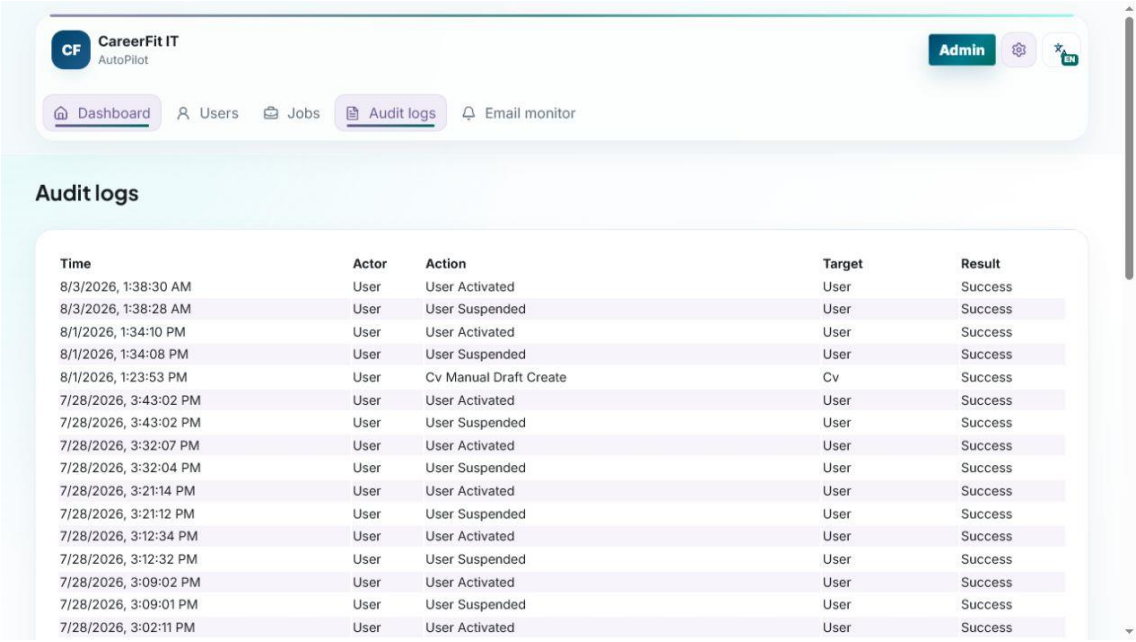
Screen 3.3. Candidate CV upload entry point



Screen 3.4. Recruiter Jobs and applicant workspace



Screen 3.5. Recruiter Talent Pool and Potential CV



Screen 3.6. Administrator audit logs

3.12 Deployment and Observability Implementation

The default local runtime starts PostgreSQL through Docker Compose and can start the backend under the backend profile. Host execution connects to localhost:5433; container execution connects to postgres:5432. The backend image mounts /app/storage/cv, and environment variables override database, JWT, CORS, application URL, mail, OCR, and storage configuration. The frontend development server runs on 127.0.0.1:5173 and uses /api or a configured API base URL.

Actuator exposes health and Prometheus endpoints. Micrometer records HTTP request metrics with the application name tag and latency histograms. Console logs include the timestamp, thread, level, request ID when available, logger, and message. These features provide basic monitoring data, but an available endpoint does not by itself prove that the monitoring system works. Chapter 4 must separately check reachability, access rules, Prometheus collection, and dashboard or alert evidence.

Maven builds the backend and can execute JUnit, Spring Security, and Testcontainers tests. The frontend build runs TypeScript checking before Vite bundling. Playwright configurations support local and production-oriented E2E runs. Build and test commands, versions, environment, failures, and generated artifacts must be recorded when Chapter 4 results are finalized.

3.13 Implementation Limitations

Table 3.6. Verified implementation limitations

Area	**Current limitation**	**Consequence or required improvement**
Content moderation	Ban/Dismiss exists, but appeal and restoration do not	Add moderator workflow, appeal, evidence retention, and abuse-rate controls
Email actions	Hashed token and confirm-then-POST flow	Add rate limiting, delivery monitoring, and deployment origin controls
CV/OCR	Preprocessing and cleanup are heuristic	Scanned layouts and low-quality images still need user correction
Text processing	Simple tokenization remains the direct-score baseline	Technology spelling and Vietnamese phrases may lose information
Potential model	Aliases and transfer weights are manually defined	Validate rules with recruiter-labeled data and review drift
Automation	Many policy guards and schedules interact	Add time-zone, quota, cooldown, deadline, and concurrency tests
Frontend session	JWT and account summary use sessionStorage	Evaluate refresh/cookie architecture and XSS controls

Area	Current limitation	Consequence or required improvement
File storage	Local path or Docker volume	Add malware scanning, encryption, backup, retention, and object storage
Audit	Several services write audit rows directly	Centralize redaction, request context, schema rules, and integrity policy

These limitations are included because the purpose of an implementation chapter is to explain the system that exists, not an idealized version. They also provide concrete hypotheses and negative cases for Chapter 4.

3.14 Chapter Summary

This chapter explained the current implementation in Spring Boot, React, PostgreSQL, and Flyway. It covered JWT security, CV review, direct matching, the Potential assessment, feedback learning, applications, Talent Pool, AutoFit, email actions, Job/CV reporting, administrator moderation, frontend integration, and monitoring. Chapter 4 evaluates the refreshed system.

CHAPTER 4. TESTING AND EVALUATION

4.1 Evaluation Objectives

The evaluation was designed to answer four questions. First, does lexical scoring with Rocchio produce repeatable results and move holdout items in the expected direction after feedback? Second, do the backend modules and database migrations pass the unit, integration, security, and contract tests? Third, can the main Guest, Candidate, Recruiter, and Administrator workflows run through the integrated browser application? Fourth, what can be measured about local API latency, authorization, health, and monitoring without presenting the results as production validation?

The evaluation treats algorithm results, software correctness, browser workflows, security checks, and system health as separate areas. Passing one area does not mean that every other area also passes. In particular, the controlled Rocchio benchmark does not measure hiring quality with real production recruitment data, and a passing backend test suite does not prove that browser, monitoring, or deployment work is complete.

Backend, algorithm, frontend build, and integrated Chrome results were refreshed on August 7, 2026 (ICT). The observed Git HEAD was 242e13a8f7d16fc9ebcab9780264c2c2b2b4ef06, but the worktree contained 209 modified or untracked entries. The results therefore describe this working tree and cannot be reproduced from the commit alone unless the full source state is preserved.

4.2 Evaluation Environment

Table 4.1. Evaluation environment

Component	Observed version or configuration
Host operating environment	Windows, PowerShell
Java	21.0.1
Backend framework	Spring Boot 3.2.5
Maven	Repository Maven Wrapper; clean verify
Node.js / npm	Bundled Node runtime with repository npm scripts
Frontend build	Vite 6.4.3, React 18.3, TypeScript 5.9, ESLint
Docker / Compose	Docker Desktop with Compose
Test database	PostgreSQL 16 through Testcontainers; local Compose PostgreSQL for E2E
Database migrations	Flyway V1–V25

Component	Observed version or configuration
Browser E2E	Playwright 1.61 using installed desktop Chrome
Integrated backend	Compose backend on port 8080
Integrated frontend	Vite development server on 127.0.0.1:5173

Docker Desktop and Compose provided PostgreSQL and the backend for integrated browser verification. Testcontainers created isolated PostgreSQL 16 databases for backend integration tests. The browser suite used the local CareerFit database, which contained imported, seeded, and E2E records. The public interface showed 992 open Jobs during the screenshot session. This runtime data is separate from the controlled algorithm dataset.

Earlier commands are recorded in evidence/CHAPTER5_EVIDENCE_20260703.md. For the August 7 refresh, Surefire XML, Maven output, TypeScript/ESLint/Vite results, Playwright output, the current report, and evaluation/result.json are the main evidence. The report states the working-tree limitation instead of treating the commit hash as a complete release identifier.

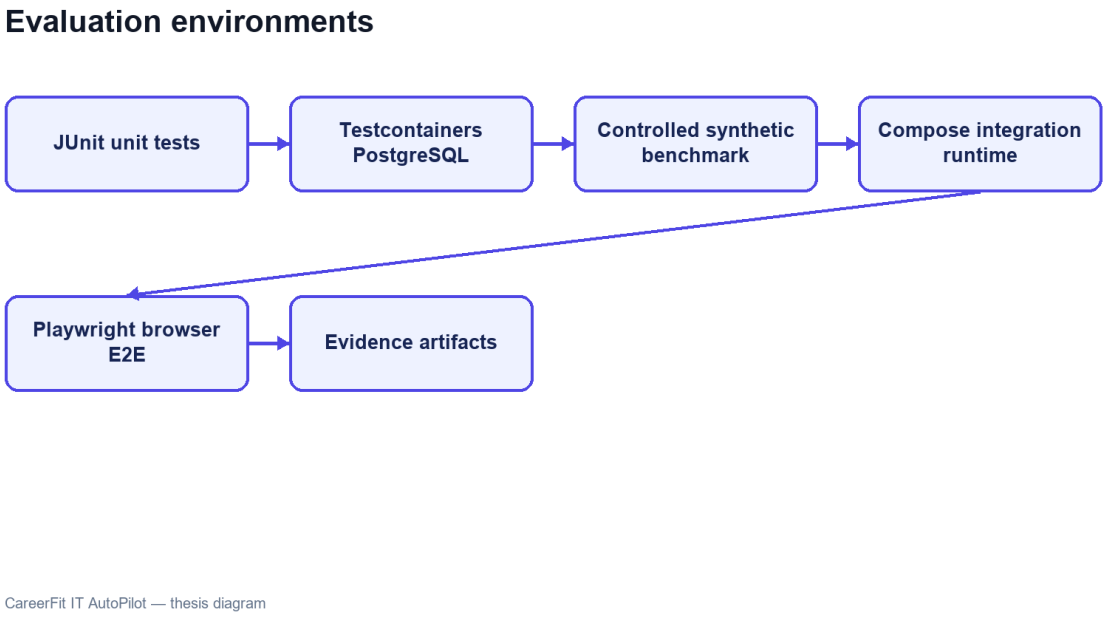


Figure 4.1. Evaluation environments and evidence sources

4.3 Dataset and Evaluation Design

4.3.1 Controlled Algorithm Dataset

AlgorithmEvaluatorTest loads evaluation/controlled-dataset.json. The dataset contains 50 Jobs and 100 unique CVs across IT domains. It defines 300 training pairs and 300 holdout pairs. The file hash observed during every repeated run was 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcf89fcc5cc2f2.

The benchmark is intentionally synthetic. Each Job is associated with a training-positive CV and a separate holdout-positive CV sharing a latent skill that is not emphasized in the original JD. The baseline ranks candidates using the original lexical representation. A positive feedback event supplies evidence from the training CV; Rocchio updates the Job vector; and the system reranks candidates. Metrics are then computed on holdout relevance rather than using the same CV that generated feedback as the only success case.

This design checks whether a defined feedback signal moves a related holdout item upward. It does not reproduce real recruiter behavior, naturally unbalanced applicant pools, demographic differences, intentionally misleading CVs, or changes in labor-market terms. The dataset includes a designed shared feature that makes a large improvement possible, so the result should not be treated as the expected effect in production.

4.3.2 Local Runtime Data

The integrated runtime used Flyway seed data and previously imported Job records. It supports realistic API volume and complete application workflows, but it is not a labeled relevance dataset. Consequently, it is suitable for functional/E2E and small latency checks, not for calculating ranking precision or fairness. E2E uses demo Candidate, Recruiter, and Administrator accounts and changes local database state.

4.4 Backend Automated Testing

The complete backend suite was executed with .\mvnw.cmd clean verify. It compiled 148 application source files and 37 test source files, started PostgreSQL 16 Testcontainers, applied Flyway migrations V1–V25, executed 35 Surefire suites, and built the backend JAR.

Table 4.2. Backend automated test results

Measure	Result
Test suites	35
Tests	141
Failures	0
Errors	0

Measure	Result
Skipped	0
Aggregated Surefire suite time	102.453 s
Maven lifecycle	clean verify and JAR build passed

The suites covered application context, API contracts, security, CV drafts and review, OCR/extraction, skill rules, Job lifecycle, catalogues, matching, feedback, automation, deadlines, bookmarks, invitations, analytics, settings, content reporting and moderation, and the controlled AlgorithmEvaluatorTest.

All 141 registered JUnit tests passed on August 7, with no failures, errors, or skips. The aggregated Surefire suite time was 102.453 seconds. Negative tests still produced expected handled errors in their own cases, but the Maven build completed successfully and no final test report contained a failure.

4.5 Controlled Algorithm Results

The benchmark used Rocchio parameters $\alpha=1.0$, $\beta=0.75$, and $\gamma=0.15$. Coverage remained 1.0 before and after feedback. Table 4.3 reports the fresh artifact values.

Table 4.3. Baseline and Rocchio results on the synthetic holdout scenario

Metric	Baseline	After Rocchio	Delta
Precision@5	0.012000	0.172000	+0.160000
Recall@5	0.060000	0.860000	+0.800000
nDCG@3	0.030000	0.830000	+0.800000
nDCG@5	0.037737	0.837737	+0.800000
nDCG@10	0.050424	0.850424	+0.800000
MRR	0.058755	0.842665	+0.783910
HitRate@5	0.060000	0.860000	+0.800000
Coverage	1.000000	1.000000	0

The increase across position-sensitive metrics shows that the controlled positive feedback moved relevant holdout CVs toward the top of the ranking. Precision@5 reaches 0.172 after feedback, so the top-five lists are not composed entirely of labeled relevant items. Recall@5 and HitRate@5 reach 0.86 in the designed scenario, while 14 percent of query cases still do not place the expected holdout item within the first five results.

The August 7 full-suite run reproduced dataset hash 6e935639ba6d3290dca8ad91a35d714c5e30c7e69a59af23ddbcf89fcc5cc2f2. It produced baseline nDCG@5 of 0.037737056145, Rocchio nDCG@5 of 0.837737056145, and a delta of 0.80. These values describe the controlled dataset and should not be read as expected improvement on real recruitment data.

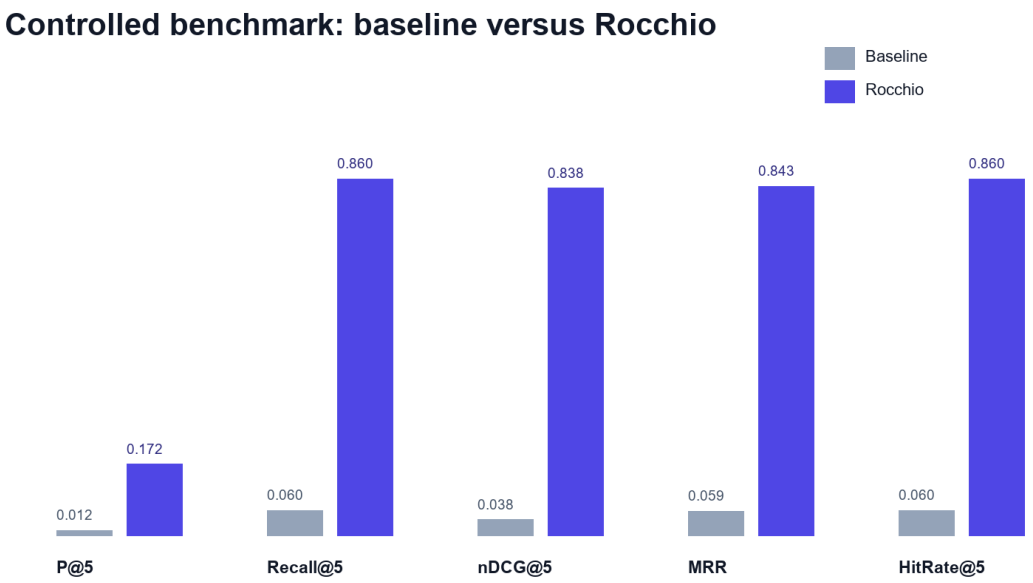


Figure 4.2. Baseline and Rocchio benchmark metrics at K = 5

4.6 Concurrency Observation in Benchmark Runs

The final isolated benchmark completed without StaleObjectStateException. Feedback learning is registered after transaction commit, and benchmark setup removes prior Matchings in bulk and clears the persistence context before recomputation. This makes the repeated baseline-versus-learned measurement deterministic within the controlled test environment.

Earlier benchmark runs showed a background database conflict even though the main test assertions passed. The current run did not log that exception because feedback learning and matching now start after the first transaction commits, and the benchmark clears previous Matching records before recalculation. This remains important: both test assertions and background errors must be checked.

The implemented correction controls asynchronous transaction timing and checks the final logs for background failures. Production work should still define retry, conflict, timeout, and recovery policies, and CI should continue to fail when uncaught background exceptions appear even if foreground JUnit assertions pass.

4.7 Frontend Build Evaluation

The frontend verification ran TypeScript checking with no output, ESLint, Vite 6.4.3 production build, and the bundle check. The August 7 build completed successfully and transformed 2,362 modules.

Table 4.4. Frontend build artifacts

Artifact	Uncompressed size	Gzip size

Artifact	Uncompressed size	Gzip size
index.html	1.37 kB	0.67 kB
Main CSS	137.80 kB	25.57 kB
Icons chunk	22.10 kB	5.03 kB
Query chunk	39.78 kB	12.20 kB
React chunk	181.35 kB	59.62 kB
Application chunk	326.95 kB	88.93 kB
Charts chunk	387.39 kB	112.63 kB

Manual Rollup chunking separated React, query, chart, and icon dependencies. The largest generated JavaScript chunk was charts at 387.39 kB (112.63 kB gzip), below Vite's 500 kB warning threshold. This is build evidence only; no browser performance profile was collected.

4.8 Browser End-to-End Evaluation

The Playwright suite ran with the Vite frontend, current backend, PostgreSQL, and installed desktop Chrome. All 46 tests across workflow, backend-contract, account-state, catalogue, and resilience coverage passed, including the three report-moderation contracts.

Table 4.5. Integrated Chrome workflow results

Scenario group	Main verification	Result
Guest and authentication	Public catalogue/detail, login, session restoration, and role routes	Passed
Candidate CV and matching	Draft, upload/review/confirm, matching, filters, and feedback	Passed
Candidate recruitment	Urgent Jobs, apply/withdraw, settings, AutoFit, and active-Job report	Passed
Recruiter workflow	Company, Job draft/publish, applicants, Talent Pool, bookmark, invitation, and visible-CV report	Passed
Administration	Suspend/reactivate, audit, report queue/detail, and ban action	Passed
Resilience and interface	API errors, navigation, account state, status labels, and dark mode	Passed

The result covers Guest search and details; email/password login; Candidate CV, matching, applications, settings, AutoFit, and active-Job reporting; Recruiter onboarding, Jobs, applicants, Talent Pool, bookmarks, invitations, and visible-CV reporting; Administrator account operations and report moderation; session restoration; catalogues; retryable errors; dark mode; and role navigation. It is not an independent UAT study, and Firefox/WebKit were not run in this refresh.

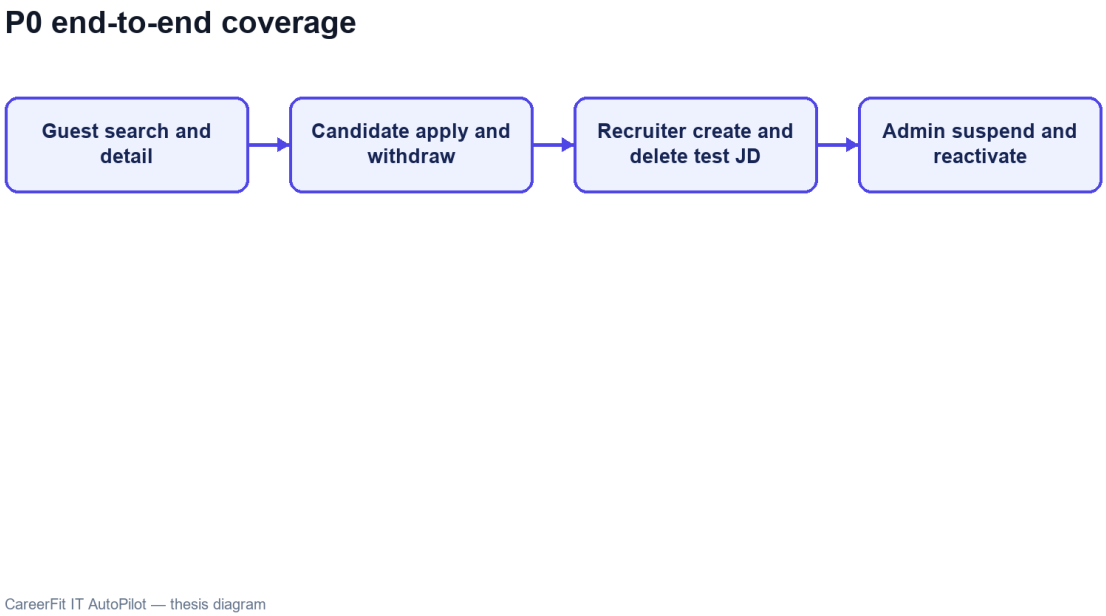


Figure 4.3. P0 end-to-end workflow coverage

4.9 Security-Oriented API Checks

Small negative checks were executed against the integrated backend to verify public access, missing authentication, invalid authorization scheme, valid Candidate authentication, and role denial.

Table 4.6. API authorization observations

Request	Expected	Observed	Result
Public Job search without token	200	200	Passed
Candidate CV endpoint without token	401	401	Passed
Current-account endpoint with invalid Basic scheme	401	401	Passed

Request	Expected	Observed	Result
Current-account endpoint with valid Candidate bearer token	200	200	Passed
Administrator dashboard with Candidate bearer token	403	403	Passed

These checks confirm selected URL-layer outcomes, not a complete penetration test. They do not evaluate token theft, cross-site scripting, CV malware, rate limiting, secret rotation, SQL-injection tooling, or every ownership boundary. The email-action flow now uses hashed storage and confirm-then-POST redemption, but the broader production-security topics remain outside the verified evidence.

4.10 Runtime Health, Monitoring, and Local Latency

The public Job API returned HTTP 200. The aggregate /actuator/health endpoint and the liveness and readiness groups returned HTTP 200 with status UP. Mail health is disabled when application mail is disabled, preventing an intentionally absent local mail provider from making aggregate health misleading. Prometheus metrics remained available in the local evaluation profile.

Table 4.7. Runtime observations

Endpoint or check	Observation
/api/jobs/search?page=0&size=20	200
/actuator/health	200, UP
/actuator/health/liveness	200, UP
/actuator/health/readiness	200, UP
/actuator/prometheus	200

The final runtime check returned HTTP 200 and status UP for aggregate health, liveness, and readiness. Mail health is disabled when application mail is disabled, so the absence of a local mail provider no longer makes aggregate health misleading. Production monitoring would still require protected component details, alerting, and an external mail check when email is enabled.

Thirty sequential warm requests to the public Job search endpoint produced a mean of 61.79 ms, p50 of 55.20 ms, p95 of 85.11 ms, minimum of 44.99 ms, and maximum of 99.32 ms. This was a single-client localhost sample without concurrent load, controlled warm-up, network latency, repeated batches, or resource monitoring. It demonstrates only that the observed local endpoint responded consistently in this small sample; it is not a throughput or capacity benchmark.

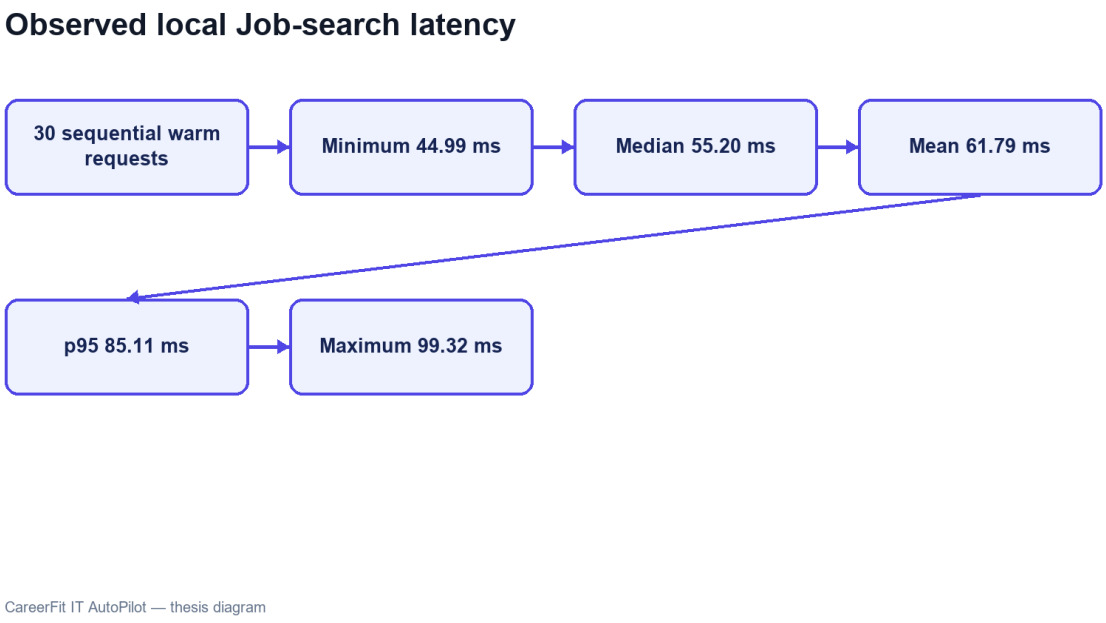


Figure 4.4. Local Job-search latency distribution

4.11 Evaluation Summary by Research Question

Table 4.8. Answers supported by current evidence

Question	Evidence-supported answer
Does Rocchio change holdout ranking in the intended direction?	Yes for the controlled synthetic dataset; the same large metric delta was reproduced.
Is the backend automated suite passing?	Yes. All 141 tests across 35 suites passed, and clean verify built the JAR.
Do the main browser workflows execute?	Yes. All 46 integrated Chrome tests passed; Firefox/WebKit and participant UAT remain incomplete.
Does report moderation work in the tested contracts?	Yes for Candidate Job report, Recruiter visible-CV report, and Administrator Job ban; production abuse handling is not evaluated.
Are selected authorization boundaries enforced?	Observed public, authentication, role, ownership, and report-visibility checks passed; this is not comprehensive security validation.
Is the runtime healthy?	Yes in the evaluated local profile: health groups and the core API returned HTTP 200.
Is performance production-ready?	Not evaluated. Only a small sequential localhost latency sample was collected.

4.12 Threats to Validity

4.12.1 Construct Validity

The synthetic benchmark measures retrieval behavior for planted relevance relationships, not hiring success, candidate quality, fairness, or recruiter satisfaction. Matching metrics cannot establish whether a person should be employed. Scripted E2E success measures workflow execution, not usability or user trust.

4.12.2 Internal Validity

The benchmark runs inside a Spring context, so asynchronous work and persisted Matching records can affect repeatability if they are not controlled. The final test registers learning after commit, clears previous Matching state before recomputation, and checks logs for background failures. Browser tests use explicit API and UI conditions, avoid relying only on the first seeded Job, and delete the Job created by the Recruiter flow.

4.12.3 External Validity

The controlled data lacks organic recruiter judgments, changing terminology, demographic diversity, adversarial behavior, and production class imbalance. The runtime is one Windows workstation with Docker Desktop, one local PostgreSQL instance, and desktop Chrome. Results cannot be generalized to enterprise load, other browsers, cloud deployment, or a population of real users.

4.12.4 Reproducibility

The dataset hash and generated JSON help reproduce the algorithm experiment. Maven Wrapper, Flyway, Testcontainers, the package lock, and recorded commands help rebuild the environment. Recruiter E2E cleanup is automatic, but exact reproduction is still harder because the working tree is not clean, the local database can change, imported records remain, and some web assets are external. The final thesis release should use a clean commit or archive together with a fixed evidence package.

4.13 Chapter Summary

The August 7 evaluation passed all 141 backend tests, completed TypeScript, ESLint, production-build, and bundle checks, and passed all 46 integrated Chrome tests. The controlled Rocchio benchmark kept its expected synthetic improvement. These results support a functioning academic prototype, but not production readiness or proven hiring effectiveness.

CONCLUSION

1. Summary of Results

CareerFit IT AutoPilot was implemented as an IT recruitment prototype with public Job discovery, role workspaces, reviewed CV processing, direct matching, a separate Potential assessment, recommendations, Rocchio feedback, applications, Talent Pool features, AutoFit policies, email actions, content reporting, administrator moderation, analytics, and audit monitoring.

The refreshed evaluation provides evidence for several technical outcomes. The backend passed 141 tests across 35 suites. TypeScript checking, ESLint, Vite production compilation, and the bundle check passed. All 46 integrated Chrome tests passed across role workflows, backend contracts, account state, catalogues, and resilience, including Candidate Job reporting, Recruiter CV reporting, and Administrator banning from the moderation tab.

The controlled Rocchio benchmark showed the expected behavior on its synthetic dataset. With 50 Jobs, 100 unique CVs, 300 training pairs, and 300 holdout pairs, nDCG@5 increased from 0.037737 to 0.837737, Recall@5 and HitRate@5 increased from 0.06 to 0.86, and MRR increased from 0.058755 to 0.842665. These results show adaptation in the designed synthetic scenario, but they do not estimate performance on real recruitment data.

The August 7 refresh synchronized the new content-reporting feature, Flyway V25, BANNED Job/CV states, administrator moderation, and the related browser contracts. The largest JavaScript chunk remained below 500 kB. The result is suitable for a local thesis demonstration, not a production release.

Table C.1. Consolidated result status

Evaluation area	Supported result	Qualification
Backend automated tests	141/141 tests across 35 suites passed	clean verify and JAR build completed
Controlled feedback benchmark	Large, deterministic Rocchio improvement	Synthetic design; not production effectiveness
Frontend checks	TypeScript, ESLint, Vite build, and bundle check passed	Largest JS chunk 387.39 kB; no Vite size warning
Browser workflows	46/46 integrated Chrome tests passed	No Firefox/WebKit or independent participant UAT
Report moderation	Job/CV reporting and Administrator Job ban contracts passed	No real moderation load, appeal, or restoration study

Evaluation area	Supported result	Qualification
Authorization spot checks	Observed 200/401/403 outcomes matched expectations	Not a complete security assessment
Runtime APIs	Core Job API and health endpoints returned HTTP 200/UP	Local profile only; no production monitoring
Local latency sample	Mean 61.79 ms and p95 85.11 ms for 30 sequential Job queries	No concurrency, controlled load, or production network

2. Achievement of Thesis Objectives

2.1 Role-Based Recruitment Platform

The role-based workflow objective was achieved at prototype level. Guests can browse public Jobs and employers. Candidates can review CVs, inspect matching and recommendation results, apply or withdraw, submit feedback, configure AutoFit, and report an active Job. Recruiters can manage Jobs, applicants, and the Talent Pool and can report a CV that is visible through an owned Job. Administrators can review grouped report cases and choose Dismiss or Ban. The 46 Chrome tests cover representative flows but not every route or browser.

2.2 CV and Job-Description Processing

The project implements uploaded and manually created CVs, explicit processing statuses, file-type and magic-byte validation, PDF and DOCX extraction, image and scanned-PDF OCR fallback, sparse-text warning, failure reason persistence, and structured quality signals. Job creation includes structured fields and conditional validation. This objective is supported by source inspection and automated tests for extraction, validation, Job service behavior, and integration contracts.

The implementation is limited by local storage, the availability of external Tesseract software, page and timeout limits, and a simple deterministic tokenizer. It should be described as a working ingestion pipeline rather than a general document-understanding system.

2.3 Matching and Recommendation

CareerFit implements a static-corpus TF-IDF vectorizer, cosine similarity, normalized score, LOW/MEDIUM/HIGH labels, potential heuristics, and shared-term reasons. CV–JD matching and profile-oriented recommendation remain separate workflows, and Matching state remains separate from Application state. Scoring, TF-IDF, batch isolation, and API contract tests passed.

The objective was achieved as an interpretable lexical baseline, not as a semantic matching model. Technology names with punctuation, synonyms, Vietnamese phrases, career changes, and context beyond shared terms remain limitations. The displayed percentage is a normalized similarity score, not a tested probability of recruitment success.

2.4 Feedback Learning

The system records GOOD_MATCH, POTENTIAL, BAD_MATCH, and NOT_INTERESTED feedback with actor and channel information. Positive and negative learning signals update the Job representation using Rocchio with $\alpha=1.0$, $\beta=0.75$, and $\gamma=0.15$. Recalculation begins from the original Job vector and current feedback history, which produces repeatable results for the same inputs. Affected Matchings are marked stale and rescored asynchronously.

The controlled benchmark confirms the intended Rocchio behavior in the planted latent-skill scenario. The final run completed without the earlier optimistic-lock exception after learning was moved to an after-commit boundary and benchmark state was cleared before recomputation. nDCG@5 and HitRate@5 each increased by 0.80, although this does not prove reliability under production concurrency.

2.5 Policy-Driven Automation and Email Action

AutoFit policies store thresholds, enablement, notification preferences, cooldown, quota, quiet-hour, timezone, digest, and pause settings. Scheduled services recompute stale results, send digests and high-match notifications, clean expired email actions, and execute auto-apply. Auto-apply validates the default CV, Job status, threshold, and duplicate state and limits creation to three applications per run.

This objective is achieved as configurable prototype automation. Notification policy and application authorization remain separate control paths, scheduler timing is externalized through app.scheduler.*, and the email-action channel uses hashed token storage with GET confirmation followed by POST execution. Further production work is still required for rate limiting, secret rotation, mail delivery, and deployment-specific origin controls.

2.6 Auditability and Evaluation

Major application, feedback, automation, content-report, moderation, and administrative actions create structured audit rows. Notification delivery, email-action status, report resolution, and target BANNED state add operational evidence. The project includes unit, integration, security, algorithm, and E2E tests and produces a dataset-hashed benchmark artifact.

Audit coverage has not been formally proven for every state-changing endpoint, and direct repository calls can still produce inconsistent metadata conventions. The evaluation nevertheless checked logs and side effects in addition to test exit codes, which helped detect and correct the earlier concurrency and health problems.

Table C.2. Objective assessment

Objective	**Assessment**	**Primary evidence**
Role-based web platform	Achieved for prototype scope	API contracts and 46 integrated Chrome tests
Reviewed CV/JD ingestion	Achieved for supported formats and configured OCR	Draft/review/confirm, extraction, quality, Job, and integration tests
Explainable matching and Potential	Achieved as rule-based baseline	TF-IDF/scoring tests and skill-transfer reasons
Rocchio feedback adaptation	Achieved in controlled test	Synthetic benchmark, after-commit execution, and clean reports
AutoFit and communication	Achieved as configurable prototype	Policies, scheduler, reminders, notifications, and signed email actions
Recruiter operational workflow	Achieved for prototype scope	Company, Job, applicants, Talent Pool, bookmark, and invitation tests
Content reporting and moderation	Achieved for the implemented basic flow	ContentReportServiceTest and three browser contracts
Production readiness	Not demonstrated	Local health passed; scale, real users, appeals, and production security remain missing

3. Discussion

3.1 Value of an Interpretable Baseline

CareerFit uses a lexical vector-space baseline instead of assuming that a dense or generative model is always better. This choice lets users inspect the input tokens, weights, shared terms, feedback centroids, and score calculation. For an academic system, this transparency helps explanation, debugging, boundary testing, and presentation of the implementation.

The cost is reduced semantic capability. A model that cannot reliably equate related technologies or interpret career context may under-rank suitable candidates and over-rank documents containing repeated keywords. The correct conclusion is

not that TF-IDF is sufficient for all recruitment, but that it provides a reproducible baseline against which future semantic or hybrid retrieval can be compared.

3.2 Human-in-the-Loop as System Structure

Human-in-the-Loop in CareerFit is expressed through multiple control points: users supply and validate data, inspect reasons, configure policy, apply or invite, provide feedback, pause automation, and review audit history. The system distinguishes score evidence from Application state and policy action. This gives users more control than adding a general approval button after an automated decision that they cannot understand.

However, human oversight does not automatically make a system fair or safe. Users may accept misleading explanations, policies may use unsuitable thresholds, and feedback may repeat individual or organizational bias. HITL must therefore be combined with error analysis, access control, ways to question or appeal results, audit review, and evaluation with representative data.

3.3 Meaning of the Rocchio Improvement

The large benchmark delta is expected from the experimental design: a latent skill is deliberately shared between a training-positive CV and a holdout-positive CV but omitted from the initial JD emphasis. Rocchio moves the Job vector toward the training example, making the holdout example easier to retrieve. Repeated equality of the metrics shows implementation determinism under this scenario.

The result does not show that every real Candidate feedback event improves ranking by 0.80 nDCG@5. Organic feedback may be inconsistent, sparse, biased, delayed, or based on salary and location rather than technical relevance. Production evaluation would require independently labeled data, time-based splits, multiple reviewers, disagreement analysis, and online or offline comparison against a baseline without feedback.

3.4 Test Results versus Background Errors

The earlier background StaleObjectStateException showed why test counts cannot be the only release check. JUnit assertions can pass while asynchronous tasks log failures outside the main test flow. Component health and overall health must also be read together. A reliable system report should check logs, health components, output files, and side effects in addition to exit codes.

Future CI should continue to fail on uncaught background exceptions, control schedulers during algorithm tests, wait for asynchronous work explicitly, and archive logs as first-class results. Operational health details should be protected while remaining available to authorized operators.

3.5 Product Positioning

CareerFit should be positioned as an IT-focused, explainable, policy-controlled academic recruitment prototype. It demonstrates how Job portal functions, ranking, recommendation, feedback, automation, email actions, and audit can be connected. It should not be positioned as having better general-purpose AI than commercial recruiter platforms or as replacing professional recruitment judgment.

The clearest difference is workflow control: matching and recommendation are separate, policies are explicit, users keep control points, feedback has a visible mathematical effect, and actions can be audited. This difference is limited in scope but is well supported by the implementation.

4. Limitations

4.1 Data and Algorithm Limitations

The controlled dataset is synthetic and designed to show how feedback changes ranking. It does not include normal language variation, recruiter disagreement, demographic analysis, misleading CVs, or changing market terms. TF-IDF still uses a static IT corpus and simple tokenization. The Potential assessment now uses a versioned skill-transfer knowledge base, but its aliases, transfer weights, family rules, and guard thresholds are still manually defined.

4.2 Evaluation Limitations

The 141 backend tests do not prove complete path coverage. Browser evaluation covers 46 automated cases in desktop Chrome only, and no independent users participated. The new moderation flow has functional and contract tests, but it has not been tested with real abuse volume, moderator teams, appeals, or fairness review. Concurrency, capacity, cross-browser behavior, accessibility with users, and real hiring outcomes remain unevaluated.

4.3 Security and Privacy Limitations

Notification action tokens are hashed, and state changes require POST after confirmation. Access tokens are stored in frontend sessionStorage rather than persistent localStorage. Local CV storage lacks documented malware scanning, encryption, retention enforcement, and recovery testing. Public Swagger and Prometheus access are suitable for local demonstration but should be restricted in production. Privacy, fairness, and management of users' personal data have not been validated with a real deployment population.

4.4 Reliability and Maintainability Limitations

The current implementation includes after-commit matching, CV review before scoring, versioned Potential rules, server-side catalogues, recruiter company onboarding, Job drafts, bookmarks, policy guards, and content-report moderation. Audit records are still written directly by several services, and the evaluated worktree is not a fixed release commit. Moderation currently supports ban or dismiss but not an appeal or restoration workflow.

Table C.3. Principal limitations and impact

Limitation	**Impact on conclusions**
Synthetic benchmark	Supports causal behavior only, not real hiring effectiveness
Lexical direct score	Limits semantic and contextual understanding
Manual skill-transfer knowledge base	Potential rules require labeled validation and maintenance
Basic moderation workflow only	Ban/Dismiss is implemented, but appeal, restoration, and real abuse load are not evaluated
Production concurrency not evaluated	Conflict, retry, and recovery behavior remain unproven at scale
Local-only health verification	Does not establish deployed availability
Chrome-only 46-test automation suite	Limits cross-browser and real-user usability conclusions
Email delivery and abuse controls not production-tested	Signed actions and reports exist, but real delivery and rate limits remain unverified
Dirty worktree and mutable E2E database	Weakens exact experiment reproduction

5. Future Work

Future security work should add rate limiting, stronger session handling, protected management endpoints, malware scanning, encrypted CV storage, retention rules, and tested backup recovery. Content moderation should add an appeal/restoration workflow, moderator assignment, evidence retention, abuse-rate limits, and tests with concurrent reports. Email delivery should be tested with a real provider, including expiry, replay, scanner behavior, and failure recovery.

The matching baseline can then be extended using a hybrid approach. Domain-aware tokenization and aliases should first address punctuation-sensitive technologies and Vietnamese phrases. Sparse lexical scores can be combined with sentence or skill embeddings and structured constraints. Any new model should be

evaluated against the current baseline on a frozen, independently labeled dataset rather than assumed superior because it is more complex.

Evaluation should expand to recruiter-labeled CV–JD pairs, temporal holdout sets, inter-rater agreement, hard-negative analysis, subgroup/fairness checks, and calibration of labels. A controlled user study should measure task completion, time, comprehension of explanations, trust calibration, and ability to override automation. Cross-browser E2E, load testing, failure injection, backup/restore, email delivery, and security testing should be included in release evidence.

Operational development should move CVs to protected object storage, define encryption and retention policy, centralize audit generation and redaction, strengthen token/session architecture, and associate each release with a clean commit and a fixed evidence archive. Integration with external ATS or Job boards should occur only after consent, ownership, error recovery, and audit contracts are defined.

6. Closing Remarks

CareerFit demonstrates controlled IT recruitment automation by connecting job discovery, explainable matching, profile-based recommendations, feedback learning, policy-driven actions, content reporting, administrator moderation, and audit records. It keeps scores, business state, reported-content state, and user decisions visible.

Local evaluation passed 141 backend tests, the frontend type/lint/build/bundle checks, 46 integrated Chrome tests, health checks, and the controlled Rocchio benchmark. This supports a tested Human-in-the-Loop prototype, not a system ready to make real hiring decisions.

REFERENCES

[1] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” Communications of the ACM, vol. 18, no. 11, pp. 613–620, 1975, doi: 10.1145/361219.361220.

[2] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.

[3] C. Buck and P. Koehn, “Quick and reliable document alignment via TF/IDF-weighted cosine distance,” in Proceedings of the First Conference on Machine Translation, vol. 2, Berlin, Germany, 2016, pp. 672–678, doi: 10.18653/v1/W16-2365.

[4] J. J. Rocchio, “Relevance feedback in information retrieval,” in The SMART Retrieval System: Experiments in Automatic Document Processing, G. Salton, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1971, pp. 313–323.

[5] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” ACM Transactions on Information Systems, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.

[6] N. Drushchak and M. Romanyshyn, “Introducing the Djinni Recruitment Dataset: A corpus of anonymized CVs and job postings,” in Proceedings of the Third Ukrainian Natural Language Processing Workshop, Torino, Italy, 2024, pp. 8–13, doi: 10.63317/2dcvy45ws6yb.

[7] X. Yu, R. Xu, C. Xue, J. Zhang, X. Ma, and Z. Yu, “ConFit v2: Improving resume-job matching using hypothetical resume embedding and runner-up hard-negative mining,” in Findings of the Association for Computational Linguistics: ACL 2025, Vienna, Austria, 2025, pp. 12775–12790, doi: 10.18653/v1/2025.findings-acl.661.

[8] S. Vaishampayan et al., “Human and LLM-based resume matching: An observational study,” in Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, NM, USA, 2025, pp. 4823–4838, doi: 10.18653/v1/2025.findings-naacl.270.

[9] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.

[10] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, Barcelona, Spain, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.

[11] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.

[13] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2006, doi: 10.6028/NIST.SP.800-92.

[14] A. Colas, J. Araki, Z. Zhou, B. Wang, and Z. Feng, “Knowledge-grounded natural language recommendation explanation,” in Proceedings of the 6th BlackboxNLP Workshop, Singapore, 2023, pp. 1–15, doi: 10.18653/v1/2023.blackboxnlp-1.1.

APPENDICES

Appendix A. Requirement-Test Traceability Matrix

The core traceability chain is: authentication and role controls to SecurityHardeningTest and P0 login flows; CV ingestion to service and integration tests; matching and feedback to AlgorithmEvaluatorTest; Job lifecycle to Recruiter flows; content reporting to ContentReportServiceTest and the three report-moderation browser contracts; administration to moderation and account-state workflows; and operations to Actuator health, build, and runtime evidence.

Appendix B. Selected API Contracts

Representative endpoints are POST /api/auth/login; GET /api/jobs/search; POST /api/cv/upload; GET/PATCH /api/cv/{cvId}/review; POST /api/matches/{matchingId}/feedback; POST /api/applications; POST /api/reports/jobs/{jobId}; POST /api/reports/cvs/{cvId}; GET /api/admin/reports; POST /api/admin/reports/{type}/{targetId}/ban or /dismiss; PATCH /api/automation/policy; and GET /actuator/health. Protected routes require a signed JWT and backend role/ownership checks.

Appendix C. Data Model Summary

Identity data is centered on UserAccount and role profiles. Recruitment data includes Job, CV with review and report fields, Matching, Application, RecruiterCvBookmark, Invitation state, Feedback, and Skill. Automation and operations use AutomationPolicy, EmailAction, Notification, DeliveryLog, ContentReport, AuditLog, and scheduler state. Flyway V1–V25 records schema changes, and one-time email action tokens are stored as SHA-256 hashes.

Appendix D. Evaluation Summary

The final evidence package contains the 141-test backend reports, controlled algorithm benchmark, frontend type/lint/build results, 46-test Chrome output, runtime health evidence, the updated report, and evaluation/result.json. Chapter 4 reports what this evidence supports and what it does not support.

Appendix E. UAT and Demonstration Script

Preparation

1. Start PostgreSQL, the backend, and the frontend; verify aggregate health and the public Job API; and prepare separate Candidate, Recruiter, and Administrator accounts.
2. Record the current Git commit, working-tree status, configuration profile, database source, and evaluation artifact locations.
3. Use clearly marked test records and note the cleanup action for every Job, CV, application, invitation, or policy created during the demonstration.

Execution

1. As a Guest, search for an IT Job, apply filters, and open Job and employer details.
2. As a Candidate, sign in, upload or draft a CV, review and confirm it, inspect matches, apply or withdraw, submit feedback, and report one designated active test Job with a clear reason.
3. As a Recruiter, complete the company profile, publish a test Job, inspect applicants and Talent Pool, bookmark and invite a Candidate, update application status, and report one visible test CV using the owned Job context.
4. Configure Candidate AutoFit thresholds, cooldown, quota, notification options, quiet hours, and human-control settings; then use run-now only for a controlled verification.
5. As an Administrator, inspect users, audit records, and the Report moderation tab; open the designated test report case; dismiss it or ban only disposable test content; then verify status, counters, and audit evidence.
6. Show Actuator health, the archived backend and Chrome test results, the benchmark artifact, and the final cleanup state.

Acceptance and cleanup

1. Record each expected and observed result, preserve any failure evidence, and do not convert an unverified step into a passing claim.
2. Delete or deactivate demonstration data, restore account state, and confirm that background logs contain no unhandled exception.
3. Archive the final evidence with a clean commit or a complete source-state package.

Appendix F. Local Deployment Instructions

Prerequisites

- Install Java 21, Node.js with npm, Docker Desktop with Docker Compose, and Git.
- Provide environment-specific database, JWT, CORS, application URL, mail, OCR, and storage configuration. Do not commit production secrets.

Startup procedure

1. From the repository root, start PostgreSQL with `docker compose up -d postgres` and wait for the container health check to pass.
2. From `Backend/careerfit-backend`, run `mvnw.cmd spring-boot:run`. The host backend connects to Compose PostgreSQL at `localhost:5433`.
3. From `Frontend`, run `npm install` when dependencies are not present, followed by `npm run dev`. The local Vite application is available at `http://localhost:5173`.

Verification

1. Open `http://localhost:8080/actuator/health` and verify the expected aggregate, liveness, and readiness responses for the selected profile.
2. Open the frontend, verify public Job search, sign in with a designated test account, and confirm authenticated API access.
3. If OCR, mail, or another optional dependency is disabled, record the limitation instead of treating the missing external service as a verified production integration.

Shutdown and cleanup

1. Stop the frontend and backend processes, then use `docker compose down` to stop the local containers. Remove volumes only when test data loss is intended.

Appendix G. Role-Based User Guide

Guest

Open the Jobs page, search by title, skill, or company, select filters, and open a Job card to view its public details. Authentication is required before application or personalized scoring.

Candidate

Sign in, open My CVs, upload a PDF/DOCX/image or save a manual draft. Review and confirm the sections before scoring. Then choose the default CV, inspect match and Potential reasons, manage applications, provide feedback, configure AutoFit, and use Report on an active Job only when there is a real issue.

Recruiter

Sign in to the Recruiter workspace, complete the company profile, and create or publish Jobs. Use applicants and Talent Pool to review visible CVs, bookmark or invite a Candidate, and update valid application states. A visible CV can be reported only with an owned Job that gives the Recruiter access to it.

Administrator

Use the administrative workspace to inspect users, Jobs, audit records, notifications, email actions, and Report moderation. Review the report reasons and target details before choosing Dismiss or Ban. Banning is a state-changing action and should be tested only on disposable records.